Risk Subgraph Extraction in Cyclic Document Graphs with Structure-Aware Monte Carlo Search

Anonymous ACL submission

Abstract

Long documents such as contracts, statutes, software package metadata, and corporate filings often contain risks that emerge through mutually dependent records. LLM-guided Monte Carlo tree search (MCTS) can improve structured document reasoning, but standard tree rollouts become unstable on strongly connected components (SCCs), where the same logical record may be revisited indefinitely. We propose SA-MCGS, a structure-aware Monte Carlo graph search method that localizes SCCs, performs local LLM rollouts with relation-first evidence memory, and collapses each cyclic region into an inspectable risk subgraph. We evaluate a locked critical structural benchmark covering four authority-backed domains: Debian dependencies, SEC EX-21 subsidiary disclosures, German Civil Code cross-references, and CUAD contracts. Across five LLMs, SA-MCGS raises mean Root@3 and Risk-all over local evidence aggregation (LEA) by 6.75 and 15.25 percentage points, respectively. The corresponding gains over one-shot prompting are 46.50 and 39.00 points. Returned subgraphs remove 55.70% of SCC nodes on average. Expert evaluation further supports the quality of the extracted evidence. Together, these results show that SA-MCGS improves risk localization and evidence completeness while producing compact subgraphs for human inspection.

1 Introduction

Many high-stakes document analysis tasks are graph problems hidden inside text (Hendrycks et al., 2021; Koreeda and Manning, 2021; Coupette et al., 2021). In contracts, one clause may define a deadline, another may suspend it, and a later amendment may restore only part of the obligation. Similar dependencies appear when statutory exceptions are narrowed by distant cross-references (Coupette et al., 2021; Holzenberger et al., 2020), or when software packages preserve old interfaces while dependent packages require newer ones to be present first (The Debian Policy Mailing List, 2025a,b). Although each record can look plausible in isolation, the risk appears only when the directed dependencies among records are followed.

Large language models (LLMs) can read these records in a single context, but context length alone does not ensure reliable retrieval, multi-hop tracing, or evidence aggregation (Liu et al., 2024; Hsieh et al., 2024; Bai et al., 2024). Following dependencies within strongly connected components (SCCs) can revisit the same records. One-shot prompting must jointly rank risk sources and assemble supporting evidence from the entire component in one generation. Complete evidence retention is therefore a distinct objective from locating a suspicious record.

MCTS supports game playing (Silver et al., 2016) and LLM reasoning (Zhou et al., 2024; Song et al., 2025). Standard MCTS unrolls trajectories as trees. In a loopy graph, this makes the search state grow with walk prefixes instead of physical records. In an SCC, even a small set of records can therefore induce many repeated path copies. Related MCTS analyses describe this as a transposition, graph-history, and loop-induced uncertainty problem (Childs et al., 2008; Kishimoto and Müller, 2004; Saffidine et al., 2012; Moerland et al., 2020). Prior Monte Carlo graph search shares transpositions in acyclic search graphs (Czech et al., 2021), while feedback-edge methods obtain directed acyclic graph (DAG) approximations by removing or reorienting edges (Eades et al., 1993; Berger and Shor, 1990). Risk extraction requires a search representation that preserves cyclic relations while consolidating evidence about the same physical records.

We propose **SA-MCGS**, a structure-aware Monte Carlo graph search method that performs local LLM rollouts within each SCC. A record's priority depends on how its constraints, excep-

tions, or handoffs interact with accumulated evidence. Relation-first memory preserves this context across rollouts and guides exploration of implicated records and relations. After search, evidence-guided core selection integrates accumulated node and relation evidence into a compact risk subgraph for inspection (Figure 1).

Our contributions are:

- A formulation of *structural risk subgraph extraction*: retaining repair-relevant evidence in compact subgraphs of cyclic document regions.
- SA-MCGS, an adaptive search method combining upper confidence bound (UCB) selection with progressive relation priors, critical-pair revisiting, and evidence-gated core pruning.
- A fixed four-domain benchmark comparing SA-MCGS with one-shot prompting and LEA across five LLMs, with component ablations, inference-cost measurements, and expert evaluation. SA-MCGS achieves higher mean Root@3 and Risk-all than either baseline.

2 Task Formulation

Record graph. Let $G = (V, E)$ be a directed graph whose nodes are document records and whose edges denote typed dependencies such as reference, constraint, modification, trigger, or control. A record may be a clause, statute section, package metadata entry, subsidiary disclosure, or other domain unit. We use Tarjan's algorithm (Tarjan, 1972) to decompose G into strongly connected components. An SCC has node set $S \subseteq V$ and induced graph $G_S = (S, E_S)$, with every node reachable from every other node.

Structural risk. We study relational risks whose evidence extends beyond suspicious wording. The core evidence contains a root record, a witness record, and sometimes affected records. The root and witness can each be locally plausible, but jointly create an incompatible obligation, temporal condition, ownership statement, or dependency handoff.

Output. Given G_S, the system returns a node set $H \subseteq S$, identified with its induced risk subgraph, and a record ranking. We evaluate:

- **Root@3**: whether the injected root appears in the top three ranked records.
- **Risk-any**: whether H contains at least one repair-relevant risk endpoint.
- **Risk-all**: whether H retains all core risk endpoints. This criterion is strict since remediation often requires seeing both sides of the contradiction.
- **Compression**: $1 - |H|/|S|$, where higher values indicate smaller subgraphs. Compression is not optimized alone; a tiny subgraph that drops the evidence endpoints is not useful.

We report effective compression and effective OC as diagnostics in Appendix A; they are not primary success metrics. An unmodified component is a source SCC; each controlled edit produces an injected variant. Evaluating one variant with one model constitutes a model-case. Variants from the same source are not independent samples. Expert evidence and repair-material coverage are evaluated separately from the benchmark endpoint metrics.

3 Method

3.1 Why ordinary MCTS is not enough

Standard tree search treats a trajectory as a growing path, following the usual MCTS loop of selection, expansion, simulation, and backpropagation (Browne et al., 2012). In a DAG, this assumption is valid. Rollouts eventually terminate, and backpropagation updates a finite set of states. In an SCC, the same physical record can be revisited through many path copies (Childs et al., 2008). If a rollout path is $p = (v_1, \ldots, v_\ell)$, ordinary TreeMCTS indexes states by path prefix. A physical node v can therefore appear as many distinct tree states,

$$\mathcal{P}_T(v) = \{p_{\le j} : p_j = v,\ p \in \Pi_T\}, \qquad (1)$$

where j indexes positions along p and Π_T contains paths generated before budget T. With this indexing, path multiplicity governs the effective state space in place of $|S|$. Repeated revisits then consume rollout budget, increase truncation, and dilute value estimates.

Figure 2 compares four mechanisms. In the matched-budget TreeMCTS–SA-MCGS diagnostic (Table 1), TreeMCTS expands 6.4 times as many states as physical nodes in the cyclic setting, and all rollouts are truncated. SA-MCGS limits expansion to 1.0 times the component size and increases Q-spread from 0.044 to 0.371. Table 1 includes the

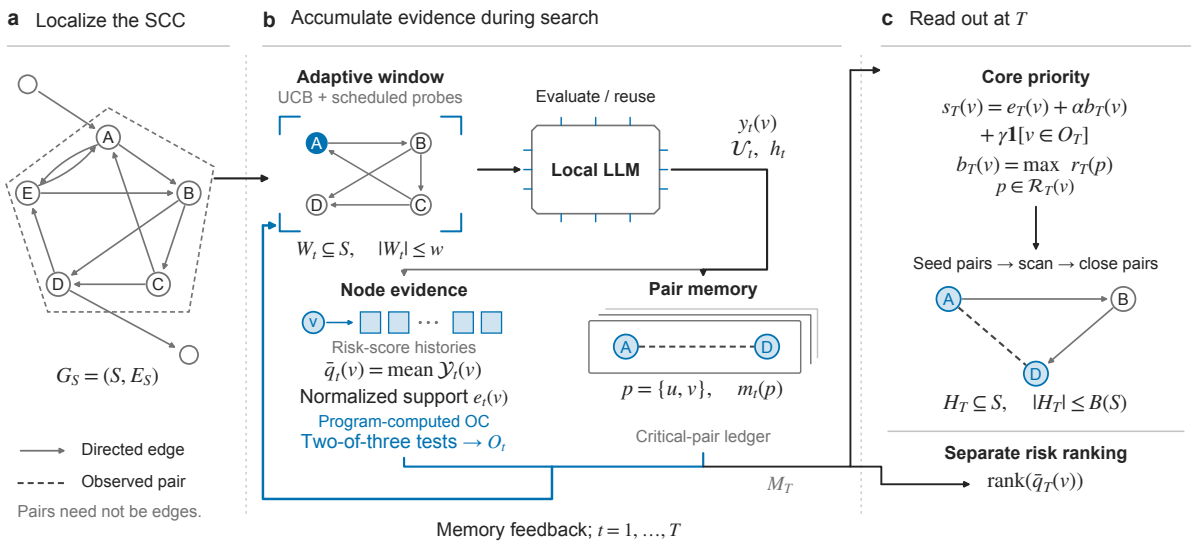


Figure 1: SA-MCGS: (a) SCC localization; (b) node/pair evidence and program-computed OC flags guide subsequent windows; (c) final core selection at T, separate from mean-risk ranking.

acyclic control; Appendix A defines the diagnostic setup and metrics.

Method	Topology	Expansion	Trunc.	Q-spread	Hit@3
TreeMCTS	DAG	2.3×	0%	0.077	7%
TreeMCTS	SCC	6.4×	100%	0.044	30%
SA-MCGS	SCC	**1.0**×	–	0.371	**90%**

Table 1: Cyclic-search ablation. Within SCC rows, bold and underlining mark best and second-best expansion (↓) and Hit@3 (↑); DAG is an acyclic reference.

3.2 Objective

SA-MCGS avoids path-copy states by treating each induced SCC graph G_S as a local search unit. The desired output is a subgraph $H \subseteq S$ that keeps enough evidence for remediation while remaining smaller than the original component. Let $R^\star \subseteq S$ denote the latent repair-relevant endpoints in evaluation, e.g., root and witness records. At inference $R^\star$ is unknown, so the algorithm estimates node and relation utility from rollout evidence. The empirical objective is given by

$$\max_{H \subseteq S} \widehat{R}(H) - \lambda_s \frac{|H|}{|S|}, \tag{2}$$

where $\widehat{R}(H)$ estimates evidence retention and λ_s controls compression pressure. The objective balances risk coverage, measured by Risk-any and Risk-all, against compression, reflecting the need for a small H to retain repair-relevant evidence.

3.3 SCC-aware local rollouts

We first decompose the record graph using Tarjan's linear-time algorithm (Tarjan, 1972). DAG regions are handled by the ordinary upstream search, whereas each non-trivial SCC is processed by SA-MCGS. Each rollout uses a window with capacity w:

$$W_t \subseteq S, \qquad |W_t| \le \min(w, |S|). \tag{3}$$

Windows use relation-biased frontier expansion from a UCB-selected seed or a scheduled endpoint probe (Appendix E). The LLM evaluator observes node text and visible directed edges restricted to W_t, excluding the remainder of the SCC. A consumed entry $\mathcal{E}_t$ contains node scores $y_t(v) \in [0, 1]$, structured pair reports, and a local subgraph h_t.

Seed selection and frontier expansion follow the UCB intuition used in MCTS (Kocsis and Szepesvári, 2006). Statistics are keyed by physical graph objects instead of path copies. Let x denote either a seed node v or a frontier edge $e = (u, v)$. SA-MCGS scores both with the same exploration template:

$$U(x) = \bar{r}(x) + c\sqrt{\frac{\log(1+N)}{n(x) + \ell(x) + \epsilon}} + \frac{\lambda_p \pi(x)}{1 + \delta\sqrt{n(x)}} + \eta(x), \tag{4}$$

All quantities use the selection-time snapshot: $\bar{r}(v) = \bar{q}(v)$ for nodes, whereas edges use mean conflict indicators; n counts visits, ℓ is weighted

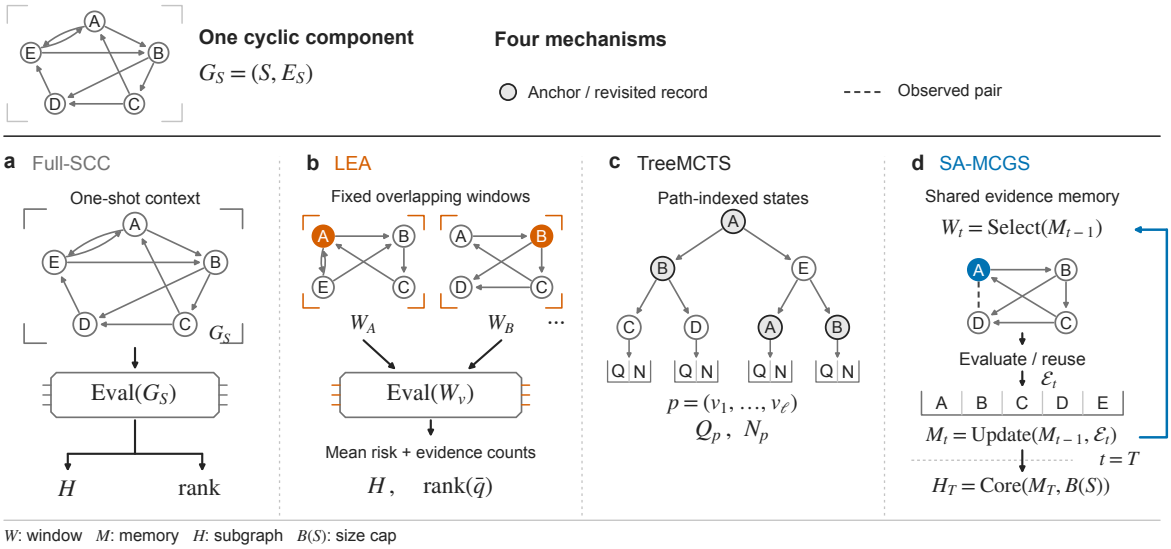


Figure 2: Four mechanisms: (a) one-shot Full-SCC; (b) fixed overlapping LEA windows; (c) path-indexed TreeMCTS; (d) SA-MCGS accumulates $\mathcal{E}_t$ and selects H_T at T.

virtual loss, π is the relation prior, λ_p its weight, and η exploration noise (Appendix E). Its normalized base component uses rollout evidence without domain-specific labels:

$$\pi_t^0(x) = \frac{[\mathbf{w}^\top \phi_t(x)]_+}{\max_{x'} [\mathbf{w}^\top \phi_t(x')]_+ + \epsilon}. \tag{5}$$

Frontier edge priors also incorporate the entering node's prior, capped at one; Appendix E gives the exact rule. For nodes, $\phi_t(x)$ contains repair-entry counts, local risk-subgraph appearances, conflict-pair endpoint support, AMAF/RAVE-style evidence, path-fragment support, and OC node flags. For edges, it contains critical-pair scores, semantic strength, repeated pair support, explicit conflict reports, reverse-edge support, and accumulated conflict evidence. Weights $\mathbf{w}$ are fixed before the locked evaluation and reported in Appendix F. The decay term in Eq. 4 prevents an early prior from dominating after sufficient node or edge samples. Together, these choices implement progressive relation bias, analogous in spirit to progressive bias and RAVE-style reuse (Gelly and Silver, 2007; Chaslot et al., 2008; Browne et al., 2012), while keeping the update key tied to physical nodes and edges as graph-level objects (Czech et al., 2021).

3.4 Relation-first evidence memory

Structural risk is relational in the sense that a node can appear harmless until previous evidence makes it incompatible with the graph context. This matches the relational-inductive-bias view that graph-structured reasoning should represent entities together with the relations that define their roles (Battaglia et al., 2018). We therefore maintain memory

$$M_t = \{n_t(v), \bar{q}_t(v), m_t(u,v), o_t(v), \ldots\}_{v,u \in S}, \tag{6}$$

Here M_t is the memory after consuming entry $\mathcal{E}_t$. For distinct nodes, $m_t(u,v) = m_t(v,u) = m_t(\{u,v\})$ counts unordered pair support; $o_t(v) = \mathbf{1}[v \in O_t]$ is the OC flag. Node risk $\bar{q}_t(v)$ is the visit-counted mean. Pair memory also retains explicit-report counts, relation reasons, and a critical-pair ledger; directed edges keep separate exploration statistics (Appendix E). At the end of search, core priority combines accumulated evidence, the strongest qualifying pair support, and an OC flag bonus:

$$\begin{aligned} b_T(v) &= \max_{p \in \mathcal{R}_T(v)} r_T(p), \\ s_T(v) &= e_T(v) + \alpha b_T(v) + \gamma \mathbf{1}[v \in O_T]. \end{aligned} \tag{7}$$

Here, $e_T(v)$ is the mean normalized evidence score, $\mathcal{R}_T(v)$ contains qualifying observed pairs involving v, and $r_T(p)$ is their pair score. We set $b_T(v) = 0$ when no pair qualifies. O_T denotes OC-flagged nodes. Critical-pair evidence contributes through $e_T(v)$ and protected-pair seeding. Appendix E specifies the calculation details.

3.5 Online evidence-discovery signal

Single LLM judgments are noisy, especially on local windows. We therefore compute online con-

flict (OC) flags from accumulated node-score histories. Once sufficient evidence is available, a node is flagged when at least two of three criteria exceed their thresholds: standardized mean risk, average empirical percentile, and the fraction of scores above the pooled median. Flagged nodes remain in OC memory, influence subsequent search priorities, and receive the bounded bonus in Eq. 7. Appendix E specifies the statistics, eligibility conditions, and thresholds. No formal coverage guarantee is claimed. Final evaluation remains based on the returned subgraph H.

3.6 Dynamic core compression

After the rollout budget, the pool $C_T^\circ = V_T^{\mathrm{ev}} \cup O_T \cup \mathrm{End}(\mathcal{Q}_T)$ combines evidence-bearing nodes, OC flags, and endpoints of supported pairs, including nonedges. Appendix E defines eligibility and the empty-pool fallback C_T. The conceptual target is:

$$H_T^\star \in \arg \max_{\substack{H \subseteq C_T \\ |H| \le B(S)}} \Bigg[\sum_{v \in H} s_T(v) + \eta_p \sum_{\substack{p \subseteq H \\ |p|=2}} m_T(p) - \lambda_c |H| \Bigg]. \tag{8}$$

We construct the returned core H_T heuristically by seeding protected and strong pairs, scanning nodes in priority order with score- and gap-based stopping, and adding supported counterparts within the cap. Equation 8 counts each unordered pair once; unobserved pairs have zero support. The conceptual pair reward η_p and size penalty λ_c are distinct from λ_s and λ_p. The paired replay in Section 6.1 isolates the effect of final-core replacement using identical search trajectories.

3.7 Computational feasibility

SA-MCGS is an anytime graph-search procedure in the sense of Zilberstein (1996). Tarjan preprocessing is $O(|V|+|E|)$. For each component, T rollout slots use windows of at most w records; eligible cache hits reuse evaluations. Input-token workload is $\sum_{j=1}^{C} L_j$ for C executed model requests with serialized prompt lengths L_j. Basic node and graph-edge statistics occupy $O(|S| + |E_S|)$ space; caches, score histories, pair ledgers, and optional traces require additional storage. The cap $B(S)$ bounds the returned core. Appendix E specifies accounting and storage. Appendix J.2 analyzes performance across rollout-budget prefixes. Section 6.2 reports call and token counts; concurrent rollouts can reduce waiting time without reducing the inference workload.

Algorithm 1 SA-MCGS for one strongly connected component

Require: Component $G_S = (S, E_S)$, budget T, window capacity w, shared rubric and domain serializer
Initialize memory M_0
for $t = 1$ to T **do**
  Select a capacity-limited window W_t using Eq. 3 and the local-window policy
  Reuse an eligible cached evaluation; otherwise query the LLM on W_t with the shared rubric
  Receive evaluation entry $\mathcal{E}_t$ with scores, pair reports, and local subgraph h_t
  Update node and relation memory M_t
  Update OC flags from accumulated node-score histories
end for
Select final core H_T from accumulated memory using Algorithm 5
return H_T and node ranking by mean observed risk $\bar{q}_T(v)$

4 Experimental Setup

4.1 Domains

We evaluate on Debian dependencies, SEC EX-21 subsidiary disclosures, German Civil Code (BGB) cross-references, and CUAD contracts, chosen for structural diversity and source authority. Appendix I summarizes their roles and graph construction.

4.2 Domain formatting and shared risk semantics

All methods receive the same visible record text and directed edges under a shared risk rubric covering structural inconsistency, mutual incompatibility, underspecification, and high-impact risks. Domain formatters preserve these risk semantics without revealing injected endpoints; Appendix I details their interface and graph construction.

4.3 Critical structural injection

Each injected variant modifies existing records in a real SCC without adding nodes. The 20 source SCCs each contribute four variants, yielding the same 80 cases per model. Five models and three methods produce 400 model-cases and 1,200 outputs. Inputs, injection identities, prompt schemas, and method settings are fixed throughout. Appendix I provides the injection templates, and Appendix K.1 specifies the run configuration.

Model	Method	Root@3	Risk-any	Risk-all	Compression
GPT-4o	One-shot	26.25	55.00	15.00	**67.89**
	LEA	78.75	96.25	43.75	51.26
	SA-MCGS	**86.25**	**100.00**	**75.00**	51.26
Gemini 2.5 Pro	One-shot	**90.00**	97.50	**85.00**	**74.98**
	LEA	87.50	**100.00**	83.75	51.26
	SA-MCGS	87.50	97.50	83.75	55.86
Qwen2.5-72B	One-shot	2.50	46.25	13.75	42.69
	LEA	67.50	**97.50**	43.75	51.26
	SA-MCGS	**70.00**	**97.50**	**75.00**	**52.82**
Llama 3.1-8B-Instruct	One-shot	3.75	47.50	25.00	30.27
	LEA	43.75	92.50	35.00	51.26
	SA-MCGS	**61.25**	**95.00**	**48.75**	**51.30**
Qwen3.5-9B	One-shot	12.50	53.75	12.50	62.70
	LEA	56.25	**91.25**	**63.75**	51.26
	SA-MCGS	**62.50**	86.25	**63.75**	**67.25**

Table 2: Results on 80 matched injected variants per model (%). Bold/underline: highest/second-highest within each model, including ties. Shading identifies SA-MCGS; higher compression means a smaller core.

4.4 Baselines and models

One-shot prompting (Full-SCC). One-shot processes the entire SCC in a single LLM call, returning a node ranking and a risk subgraph.

Local Evidence Aggregation (LEA). LEA evaluates a four-record window anchored at each SCC node, using the same local evaluator schema as SA-MCGS. It averages node scores and aggregates evidence into a fixed-cap subgraph, without adaptive search, relation memory, critical-pair revisiting, or core replacement. Appendix K.1 details window construction, aggregation, and caching. SA-MCGS uses 60 rollouts and final-core selection; the comparison fixes the local output task, but not total computation.

We evaluate SA-MCGS with five LLM backbones: proprietary GPT-4o (OpenAI, 2024) and Gemini 2.5 Pro (Gemini Team, 2025), and open-weight Qwen2.5-72B (Qwen Team, 2024), Llama 3.1-8B-Instruct (Llama Team, 2024), and Qwen3.5-9B (Qwen Team, 2026).

5 Main Results

SA-MCGS improves Root@3 over both baselines on four of the five backbones, with the largest gain over LEA on Llama 3.1-8B-Instruct (17.50 percentage points; Table 2). It improves Risk-all over LEA by 31.25 points on GPT-4o and Qwen2.5-72B, and by 13.75 on Llama. On Gemini 2.5 Pro, all three methods achieve Risk-all between 83.75% and 85.00%.

Over five fixed backbones, mean Risk-all improves by 15.25 points over LEA (95% source-SCC bootstrap interval: 2.00–27.75); the Root@3 interval includes zero (−0.25–14.00). Resampling preserves source-level dependencies (Appendix K.12).

Pooling five models, SA-MCGS raises Risk-all over LEA from 26.67% to 51.67% on BGB and from 53.00% to 69.00% on CUAD (Table 14, panel B). Across all three SCC-size groups, SA-MCGS improves Risk-all over LEA for GPT-4o and Qwen2.5-72B, and improves Root@3 for Llama 3.1-8B-Instruct (Appendix J.1). Appendix C compares the three methods on the same long-SCC case.

SA-referenced budget comparison. We compare enhanced repeated-sampling LEA and Full-SCC with fixed historical SA-MCGS outputs. Across five backbones and 80 variants per backbone, recorded or calibrated SA-MCGS reference budgets total approximately 89.2 million input and output tokens (Appendix K.2). Both baselines receive the same budget for each backbone and case.

SA-MCGS achieves higher Risk-all than repeated-sampling LEA on all five backbones (Figure 3). The gains are 33.75 percentage points on GPT-4o and 25.00 points on Qwen2.5-72B; SA-MCGS reaches 75.00% on both. Kimi illustrates a different coverage–compactness balance: Full-SCC reaches higher Risk-all (52.50% versus 32.50%) but retains 86.40% of the graph on average, compared with 48.15% for SA-MCGS.

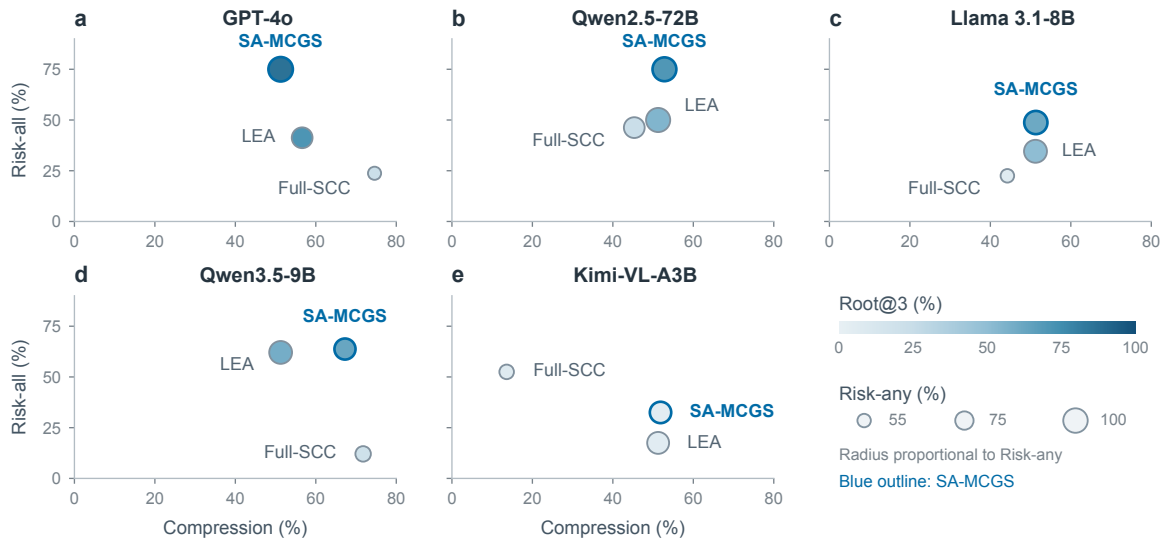


Figure 3: Five-backbone comparison with SA-referenced baseline budgets. Position shows compression and Risk-all; bubble radius is proportional to Risk-any, and fill color encodes Root@3 (0–100%). Blue outlines mark historical SA-MCGS outputs.

Configuration	Risk-all	Comp.
Stage-1 Full	**77.5**	51.3
No relation memory	73.5	51.3
No critical-pair revisit	56.3	71.3
Random local window	71.3	51.3
Stage-2 replacement core	**75.0**	51.3
Locked no-replacement	37.5	51.3
Cross-backbone replay: locked → dynamic		
Llama 3.1-8B	35.00 → **62.50**	51.26
Qwen3.5-9B	33.75 → **57.50**	66.55
Kimi-VL-A3B	30.00 → **35.00**	51.93

Table 3: Ablations (%) on 80 variants from 20 source SCCs. Stage-1 tests search components on one backbone; core replay covers four backbones.

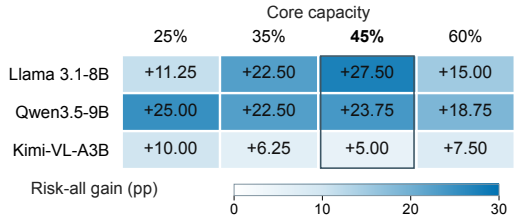


Figure 4: Dynamic-minus-locked Risk-all (pp). Outline: default cap; full results in Appendix K.6.

Method	Risk-all (%) GPT-4o	Gemini	Calls	Tokens
One-shot	25.0	**100.0**	**1.0**	**23.5**K
LEA	41.7	83.3	16.9	63.7K
SA-MCGS	**83.3**	91.7	56.5	203.4K

Table 4: Cost–performance: 12 sources, 72 runs, GPT-4o/Gemini 2.5 Pro. Strict Risk-all (↑); model-averaged calls/tokens (↓) include recovery. Bold/underline: best/second-best per column. Shading: SA-MCGS.

6 Analysis

6.1 Ablation study

Full exceeds no-memory Risk-all by 4.00 points at equal mean core size (Table 3); the comparison spans batches (Appendix K.3).

Shared-evidence replacement. Replay fixes evidence proposals and per-case core size. Replacement raises Risk-all by 5.00–27.50 points on all three further backbones (Table 3; Appendix K.5). GPT-4o replay yields a 37.50-point gain at equal mean core size (95% source-SCC paired interval: 12.50–60.00; Appendix K.4).

Capacity robustness. Matched-size gains persist in all 12 tested model–capacity comparisons (Figure 4). Appendices K.7–K.8 examine selector choice and rollout prefixes.

6.2 Cost and sensitive analysis

Table 4 reports inference costs and endpoint retention. SA-MCGS uses about 3.2 times as many tokens as LEA, with Risk-all gains of 41.7 percentage points on GPT-4o (95% CI: 8.3–75.0) and 8.3 on Gemini 2.5 Pro (−16.7–33.3). One-shot performs best on Gemini in this sample. Three invalid GPT-4o One-shot outputs remain in the denomina-

tor. Concurrent rollouts can reduce wall-clock time without reducing token use; Appendix K.9 details the parallel-time model and recovery accounting.

On four prespecified sources, varying the relation-prior bonus multiplier by $\pm 20\%$ does not reduce final Risk-all but changes Root@3. This single-coefficient check does not establish robustness to other priors or thresholds (Appendix K.10).

6.3 Expert evidence and unmodified sources

Experts select a *minimal evidence set* M, the fewest records supporting their risk judgment, and a broader *repair-material set* R with context needed to understand and repair the risk. We assess 24 injected variants and 18 unmodified sources. For the latter, a third expert independently adjudicates under fully blinded conditions whenever the first two disagree on risk existence or either reports confidence below 3/5.

On the 24 variants (14 sources), source-weighted minimal-evidence recall is 90.8% for GPT-4o SA-MCGS versus 65.4% for LEA, and 88.5% versus 86.3% for Gemini 2.5 Pro. Repair-reference precision rises from 66.5% to 85.0% and from 71.8% to 88.4%, respectively (confidence intervals in Appendix B.2).

Among the unmodified sources, experts identify 15 risk and three no-risk cases. SA-MCGS flags all 15 risk cases with both models, matching expert judgments on 18/18 (100.0%) with GPT-4o and 15/18 (83.3%) with Gemini 2.5 Pro. All errors are false alarms. These results extend evaluation beyond controlled injection, with backbone-dependent no-risk recognition.

7 Related Work

LLM reasoning and long context. Chain-of-thought prompting supports sequential reasoning (Wei et al., 2022); self-consistency aggregates answers across sampled reasoning paths (Wang et al., 2023c). Long-context and graph studies identify retrieval and encoding limitations (Liu et al., 2024; Hsieh et al., 2024; Bai et al., 2024; Wang et al., 2023a; Fatemi et al., 2024). Legal benchmarks cover contract extraction and inference (Hendrycks et al., 2021; Koreeda and Manning, 2021), mergers (Wang et al., 2023b), provision classification (Tuggener et al., 2020), unfair clauses (Lippi et al., 2019), and broader legal understanding (Chalkidis et al., 2022; Guha et al., 2023). Legal networks and contract graphs model record relationships (Coupette et al., 2021; Dechtiar et al., 2025).

Search-augmented LLMs. Tree of Thoughts (Yao et al., 2023a), Graph of Thoughts (Besta et al., 2024), and reasoning as planning (Hao et al., 2023) organize reasoning through search. ReAct interleaves reasoning and actions (Yao et al., 2023b); LATS combines MCTS with LLM evaluation, reflection, and environmental feedback (Zhou et al., 2024).

Monte Carlo search on graphs. Monte Carlo graph search shares transpositions (Childs et al., 2008; Czech et al., 2021), while related analyses address loop-induced uncertainty (Moerland et al., 2020). ReKG-MCTS (Song et al., 2025) and Graph-MCTS (Liu, 2025) apply MCTS to reasoning over external graph knowledge. SubgraphX uses MCTS to identify subgraphs that explain GNN predictions (Yuan et al., 2021).

Graph retrieval and evidence selection. RAG conditions generation on retrieved passages (Lewis et al., 2020); IRCoT interleaves retrieval and reasoning (Trivedi et al., 2023). GraphRAG summarizes communities (Edge et al., 2024), ToG-2 couples graph traversal and document retrieval (Ma et al., 2025), and G-Retriever selects query-relevant subgraphs through prize-collecting Steiner tree optimization (He et al., 2024). LegalGraphRAG combines hierarchical legal graphs, retrieval, and verification (Chen et al., 2026); SAT-Graph RAG models legal hierarchy, time, and provenance (de Martim, 2025). SA-MCGS accumulates relation evidence within SCCs to retain risk endpoints in compact subgraphs.

8 Conclusion

We introduced SA-MCGS, a structure-aware Monte Carlo graph search method for cyclic document risk. It accumulates node and pair evidence within SCCs across local windows, then selects a bounded core after search.

Across four domains and five LLMs, SA-MCGS achieves higher mean Root@3 and Risk-all than one-shot prompting and LEA. Shared-trace replay further shows that final-core replacement improves endpoint retention at the same mean core size. Expert evaluation supports evidence quality on injected variants and case-level risk detection on unmodified sources. Its compact risk subgraphs consolidate cyclic dependencies into evidence for human inspection.

Limitations

SA-MCGS focuses on risk-evidence extraction within cyclic document components. Its local LLM search incurs additional inference cost, making high-risk components a natural priority for deployment. Our evaluation spans four domains and combines controlled injection with expert assessment of 18 unmodified sources; broader natural-case coverage and end-to-end document review are directions for further evaluation. Performance depends on the backbone and graph construction, particularly for no-risk recognition and evidence completeness in long contracts. Stage-1 component comparisons cover 80 variants from 20 source SCCs on one backbone. Extending these comparisons across model families and repeated evaluations would help characterize the influence of backbone-dependent risk recognition and evidence retention. Prior sensitivity covers four sources; broader source coverage and parameter settings would provide a more complete assessment of robustness.

Ethical Considerations

SA-MCGS is intended to support human review by identifying candidate risk evidence in cyclic document components. Its outputs should be checked against the original documents by qualified reviewers and should not be used as the sole basis for legal, regulatory, or other consequential decisions.

The conflicts introduced through controlled injection are constructed evaluation cases. They should not be interpreted as factual claims about the organizations or individuals named in the source documents, or as evidence that the original documents contain those conflicts.

References

Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3119–3137. Association for Computational Linguistics.

Peter W. Battaglia, Jessica B. Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, Caglar Gulcehre, Francis Song, Andrew Ballard, Justin Gilmer, George Dahl, Ashish Vaswani, Kelsey Allen, Charles Nash, Victoria Langston, and 8 others. 2018. Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261*.

Bonnie Berger and Peter W. Shor. 1990. Approximation algorithms for the maximum acyclic subgraph problem. In *Proceedings of the First Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 236–243. Society for Industrial and Applied Mathematics.

Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. 2024. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 17682–17690.

Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. 2012. A survey of Monte Carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43.

Ilias Chalkidis, Abhik Jana, Dirk Hartung, Michael Bommarito, Ion Androutsopoulos, Daniel Martin Katz, and Nikolaos Aletras. 2022. LexGLUE: A benchmark dataset for legal language understanding in English. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4310–4330. Association for Computational Linguistics.

Guillaume M. J.-B. Chaslot, Mark H. M. Winands, H. Jaap van den Herik, Jos W. H. M. Uiterwijk, and Bruno Bouzy. 2008. Progressive strategies for Monte-Carlo tree search. *New Mathematics and Natural Computation*, 4(3):343–357.

Zerui Chen, Qinggang Zhang, Zhishang Xiang, Zhimin Wei, Linfeng Gao, Xiao Huang, Zhihong Zhang, and Jinsong Su. 2026. LegalGraphRAG: Multi-agent graph retrieval-augmented generation for reliable legal reasoning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 37455–37484, San Diego, California, United States. Association for Computational Linguistics.

Benjamin E. Childs, James H. Brodeur, and Levente Kocsis. 2008. Transpositions and move groups in Monte Carlo tree search. In *2008 IEEE Symposium On Computational Intelligence and Games*, pages 389–395. IEEE.

Corinna Coupette, Janis Beckedorf, Dirk Hartung, Michael Bommarito, and Daniel Martin Katz. 2021. Measuring law over time: A network analytical framework with an application to statutes and regulations in the United States and Germany. *Frontiers in Physics*, 9:658463.

Johannes Czech, Patrick Korus, and Kristian Kersting. 2021. Improving AlphaZero using Monte-Carlo graph search. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 31, pages 103–111.

Hudson de Martim. 2025. An ontology-driven graph RAG for legal norms: A hierarchical, temporal, and deterministic approach. In *Legal Knowledge and Information Systems: JURIX 2025*, Frontiers in Artificial Intelligence and Applications, pages 282–287. IOS Press.

Moriya Dechtiar, Daniel Martin Katz, Mari Sundaresan, Sylvain Jaume, and Hongming Wang. 2025. GRAPH-GRPO-LEX: Contract graph modeling and reinforcement learning with group relative policy optimization. *arXiv preprint arXiv:2511.06618*.

Jay DeYoung, Sarthak Jain, Nazneen Fatema Rajani, Eric Lehman, Caiming Xiong, Richard Socher, and Byron C. Wallace. 2020. ERASER: A benchmark to evaluate rationalized NLP models. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4443–4458, Online. Association for Computational Linguistics.

Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. 2018. The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1383–1392, Melbourne, Australia. Association for Computational Linguistics.

Peter Eades, Xuemin Lin, and W. F. Smyth. 1993. A fast and effective heuristic for the feedback arc set problem. *Information Processing Letters*, 47(6):319–323.

Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*.

Bahare Fatemi, Jonathan Halcrow, and Bryan Perozzi. 2024. Talk like a graph: Encoding graphs for large language models. In *The Twelfth International Conference on Learning Representations*.

Sylvain Gelly and David Silver. 2007. Combining online and offline knowledge in UCT. In *Proceedings of the 24th International Conference on Machine Learning*, pages 273–280. ACM.

Gemini Team. 2025. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*.

Neel Guha, Julian Nyarko, Daniel E. Ho, Christopher Ré, Adam Chilton, Aditya Narayana, Alex Chohlas-Wood, Austin Peters, Brandon Waldon, Daniel N. Rockmore, Diego Zambrano, Dmitry Talisman, Enam Hoque, Faiz Surani, Frank Fagan, Galit Sarfaty, Gregory M. Dickinson, Haggai Porat, Jason Hegland, and 21 others. 2023. LegalBench: A collaboratively built benchmark for measuring legal reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 36, pages 44123–44279. Curran Associates, Inc.

Shibo Hao, Yi Gu, Haodi Ma, Joshua Jiahua Hong, Zhen Wang, Daisy Zhe Wang, and Zhiting Hu. 2023. Reasoning with language model is planning with world model. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 8154–8173. Association for Computational Linguistics.

Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh V. Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. 2024. G-Retriever: Retrieval-augmented generation for textual graph understanding and question answering. In *Advances in Neural Information Processing Systems*, volume 37, pages 132876–132907. Curran Associates, Inc.

Dan Hendrycks, Collin Burns, Anya Chen, and Spencer Ball. 2021. CUAD: An expert-annotated NLP dataset for legal contract review. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, volume 1.

Nils Holzenberger, Andrew Blair-Stanek, and Benjamin Van Durme. 2020. A dataset for statutory reasoning in tax law entailment and question answering. In *Proceedings of the Natural Legal Language Processing Workshop 2020*, volume 2645 of *CEUR Workshop Proceedings*, pages 31–38. CEUR-WS.org.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. RULER: What's the real context size of your long-context language models? In *First Conference on Language Modeling*.

Akihiro Kishimoto and Martin Müller. 2004. A general solution to the graph history interaction problem. In *Proceedings of the Nineteenth National Conference on Artificial Intelligence*, pages 644–649.

Levente Kocsis and Csaba Szepesvári. 2006. Bandit based Monte-Carlo planning. In *Machine Learning: ECML 2006*, volume 4212 of *LNCS*, pages 282–293. Springer.

Yuta Koreeda and Christopher D. Manning. 2021. ContractNLI: A dataset for document-level natural language inference for contracts. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 1907–1919. Association for Computational Linguistics.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc.

Marco Lippi, Przemysław Pałka, Giuseppe Contissa, Francesca Lagioia, Hans-Wolfgang Micklitz, Giovanni Sartor, and Paolo Torroni. 2019. CLAUDETTE: an automated detector of potentially unfair clauses in online terms of service. *Artificial Intelligence and Law*, 27(2):117–139.

Lihui Liu. 2025. Monte Carlo tree search for graph reasoning in large language model agents. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*, pages 4966–4970. ACM.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.

Llama Team. 2024. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Shengjie Ma, Chengjin Xu, Xuhui Jiang, Muzhi Li, Huaren Qu, Cehao Yang, Jiaxin Mao, and Jian Guo. 2025. Think-on-Graph 2.0: Deep and faithful large language model reasoning with knowledge-guided retrieval augmented generation. In *International Conference on Learning Representations*, pages 52782–52806.

Thomas M. Moerland, Joost Broekens, Aske Plaat, and Catholijn M. Jonker. 2020. The second type of uncertainty in Monte Carlo Tree Search. *arXiv preprint arXiv:2005.09645*.

OpenAI. 2024. GPT-4o system card. *arXiv preprint arXiv:2410.21276*.

Qwen Team. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.

Qwen Team. 2026. Qwen3.5-9B. Official model card, revision c202236. Accessed October 1, 2026.

Abdallah Saffidine, Tristan Cazenave, and Jean Méhat. 2012. UCD: Upper confidence bound for rooted directed acyclic graphs. *Knowledge-Based Systems*, 34:26–33.

David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. 2016. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489.

Xiaozhuang Song, Shufei Zhang, and Tianshu Yu. 2025. ReKG-MCTS: Reinforcing LLM reasoning on knowledge graphs via training-free Monte Carlo tree search. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 9288–9306, Vienna, Austria. Association for Computational Linguistics.

Robert Tarjan. 1972. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160.

The Debian Policy Mailing List. 2025a. *Debian Policy Manual: Declaring relationships between packages*. Version 4.7.2.0, chapter 7.

The Debian Policy Mailing List. 2025b. *Debian Policy Manual: Shared libraries*. Version 4.7.2.0, chapter 8.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2023. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10014–10037, Toronto, Canada. Association for Computational Linguistics.

Don Tuggener, Pius von Däniken, Thomas Peetz, and Mark Cieliebak. 2020. LEDGAR: A large-scale multilabel corpus for text classification of legal provisions in contracts. In *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 1235–1241, Marseille, France. European Language Resources Association.

Heng Wang, Shangbin Feng, Tianxing He, Zhaoxuan Tan, Xiaochuang Han, and Yulia Tsvetkov. 2023a. Can language models solve graph problems in natural language? In *Advances in Neural Information Processing Systems*, volume 36, pages 30840–30861. Curran Associates, Inc.

Steven H. Wang, Antoine Scardigli, Leonard Tang, Wei Chen, Dimitry Levkin, Anya Chen, Spencer Ball, Thomas Woodside, Oliver Zhang, and Dan Hendrycks. 2023b. MAUD: An expert-annotated legal NLP dataset for merger agreement understanding. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 16369–16382, Singapore. Association for Computational Linguistics.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023c. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837. Curran Associates, Inc.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023a. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, pages 11809–11822. Curran Associates, Inc.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023b. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*.

Hao Yuan, Haiyang Yu, Jie Wang, Kang Li, and Shuiwang Ji. 2021. On explainability of graph neural networks via subgraph explorations. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 12241–12252. PMLR.

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. 2024. Language Agent Tree Search unifies reasoning, acting, and planning in language models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 62138–62160. PMLR.

Shlomo Zilberstein. 1996. Using anytime algorithms in intelligent systems. *AI Magazine*, 17(3):73–83.

A Diagnostic Metrics and Ablation Definitions

The main paper evaluates Root@3, Risk-any, Risk-all, and compression. We also log two diagnostics. **Effective compression** is computed only on cases whose returned subgraph retains at least one repair-relevant endpoint. It separates useful compression from trivial compression that drops all evidence. **Effective OC** at a rollout prefix indicates whether accumulated OC-flagged nodes intersect the injected risk or affected-record set. Its rate averages this cumulative indicator over cases with an available trace point at or before that prefix. It is a convergence diagnostic, not a primary success metric.

Motivating search diagnostic. We compare TreeMCTS on acyclic and strongly connected document graphs with SA-MCGS on the same SCC setting. The comparison uses matched components, the same local risk task, and equal rollout budgets. Table 1 reports state expansion, rollout truncation, Q-spread, and Hit@3, as defined below.

Table 1 uses four ablation diagnostics. **Expansion** is the number of search states expanded relative to the physical component size. **Truncation** is the fraction of rollouts stopped by the depth limit before a terminal state; loops can prevent termination (Moerland et al., 2020). **Q-spread** is the range of estimated action values at the root, a proxy for whether search can separate promising actions under UCT-style selection (Kocsis and Szepesvári, 2006). **Hit@3** reports whether a repair-relevant endpoint appears in the top three ranked records.

B Expert Annotation and Evaluation

We report the original 12-case audit below and the new 42-presentation audit in Section B.2 separately because their annotation schemes and annotator coverage differ. The original aggregate results are retained without imputing missing labels.

B.1 Original 12-case audit

We construct a representative expert-audit set to verify that the critical structural risks used in our benchmark are recognizable to qualified domain readers, not artifacts of hidden system labels. The audit is *system-blind*. Annotators see only paragraph text, paragraph identifiers, and ordinary natural-language instructions. They are not shown model outputs, hidden node roles, system suggestions, SA-MCGS cores, baseline subgraphs, or terms such as root, witness, affected node, SCC, and risk subgraph. Hidden fields are used only after annotation to compare expert labels with injected endpoints and system outputs.

All three annotators hold doctoral degrees in relevant fields and have professional experience with legal, regulatory, or contractual analysis. Table 6 summarizes the annotation tasks.

Case-level task. For each case, the expert reads a group of related paragraphs and answers whether the group contains a structural conflict. A structural conflict means that two or more paragraphs cannot all be satisfied together, leave a high-impact condition unresolved, or make a repair decision depend on inconsistent assumptions.

Paragraph-level task. For each paragraph, the expert chooses one of four labels:

- **Clear issue**: the paragraph is itself problematic, or clearly conflicts with another visible paragraph.
- **Possibly relevant**: the paragraph may be needed to understand or repair the conflict, but is not clearly wrong by itself.
- **No issue observed**: the paragraph appears to be ordinary context.
- **Uncertain**: the expert cannot decide without more context or specialist knowledge.

Repair-material task. After labeling paragraphs, the expert selects which paragraphs should be included in the final repair packet. This task is stricter than identifying a suspicious sentence because the packet should include enough material for a reviewer to understand and fix the conflict.

Annotation protocol. Each sample is annotated independently by three experts. Every sample therefore receives three case-level decisions, three sets of paragraph-level labels, and three repair-packet selections. Disagreements are preserved for analysis without being forced into an immediate majority vote.

Audit results. Table 5 summarizes the completed annotations. All 12 audited cases receive a majority structural-conflict label. Experts also recognize the injected evidence endpoints. In total, 23 of 24 root/witness endpoint paragraphs receive a majority *clear issue* label, and all 24 receive a majority *clear issue* or *possibly relevant* label. Agreement is strongest for the clear-risk endpoint decision, with Fleiss' $\kappa = 0.606$ and pairwise Cohen's κ ranging from 0.452 to 0.759. These results support that the injected critical risks are semantically visible to qualified readers when the relevant paragraphs are inspected.

The repair-packet task is intentionally stricter and more variable. One annotator selected no individual paragraph for the final packet while still marking every case as conflicting and most cases as sufficiently or partially supported. We therefore treat repair-packet selection as a conservative diagnostic of what a reviewer would include for remediation, not as the primary evidence that a conflict exists.

Role-level sanity check. After annotation, we compare expert paragraph labels with hidden roles. True endpoints are the only role class with near-universal majority recognition, with 95.8% majority clear issue and 100% majority clear-or-relevant. In contrast, affected paragraphs are rarely majority clear issue, and most broader system-core paragraphs are judged ordinary context. This behavior follows from the benchmark design, which asks methods to retain compact endpoints needed for repair, not to mark every retained context paragraph as intrinsically wrong. The same observation explains why Risk-all is strict, as a method may surface useful context or affected nodes while still missing one side of the contradiction.

The example in Table 7 illustrates the intended difficulty level. Neither entity record is a syntactic error by itself. The conflict appears because both records refer to the same consolidation path while asserting mutually incompatible ownership treatments. A risk-subgraph method should preserve this type of structural conflict because a reviewer can reliably repair the issue after seeing both endpoints.

B.2 Expanded Expert Evaluation

Samples and annotation. The expanded evaluation covers 24 injected variants from 14 source SCCs and 18 unmodified source SCCs, totaling 42 presentations and 648 paragraphs. The unmodified sources comprise five SEC, three BGB, and ten CUAD cases. Two experts annotate risk existence, material sufficiency, confidence on a five-point scale, and the evidence sets defined in Section 6.3. The unmodified-source annotations were updated through a partial re-review; the 24 injected-variant annotations are retained. The original 12-case audit uses a separate annotation scheme and is reported above.

For the unmodified-source review, a third expert makes the final decision if the first two experts disagree on risk existence or either confidence score is below 3. The third expert conducted the adjudication independently under fully blinded conditions. The final unmodified-source labels contain 15 risk cases and three no-risk cases. Risk-existence evaluation includes all 18 cases; sufficiency is assessed separately. Annotation records and final decisions are retained with the accompanying data.

Table 8 separates agreement on risk existence from agreement on material sufficiency. On the 24 injected variants, mean evidence-set Jaccard similarity between the first two experts is 0.670 for minimal evidence and 0.723 for repair materials. Third-expert judgments are used for targeted adjudication rather than treated as an independent full-sample agreement estimate.

Expert-reference scoring. Following ERASER's distinction between human-rationale agreement and faithfulness to model predictions (DeYoung et al., 2020), we assess agreement with expert-selected evidence. For a returned subgraph H, minimal-evidence recall is $|H \cap M|/|M|$, repair coverage is $|H \cap R|/|R|$, and repair-reference precision is $|H \cap R|/|H|$. The outside-reference fraction, $|H \setminus R|/|H|$, measures disagreement

Audit quantity	Result	Interpretation
Audited critical cases	12 cases, 36 case-level decisions	Each case is independently labeled by three experts.
Majority structural conflict	12/12 (100%)	The audited critical cases are judged to contain structural conflicts.
Repair material sufficiency	10 yes, 1 partial, 1 tie	Most cases contain enough visible material to support repair, but one case remains divided.
No extra context needed	10/12 (83.3%)	Most cases are understandable from the provided paragraphs alone.
Mean confidence	4.28/5	Experts are generally confident in their case-level judgments.
Endpoint majority clear issue	23/24 (95.8%)	Root/witness endpoints are usually judged directly problematic.
Endpoint majority clear-or-relevant	24/24 (100%)	Every endpoint is recognized as at least repair-relevant.
Clear-issue agreement	Fleiss' $\kappa = 0.606$	Agreement is substantial for the binary endpoint-risk judgment.
Repair-packet endpoint-any	7/12 (58.3%)	Majority repair packets include at least one endpoint in over half of cases.
Repair-packet endpoint-all	6/12 (50.0%)	Selecting all endpoints is stricter and less stable across experts.

Table 5: System-blind expert audit results. Experts did not see system outputs, hidden roles, or model-generated hints during annotation.

Task	Expert-facing question	Example expert note
Case-level conflict	Do these paragraphs create a conflict or unresolved high-risk condition when read together?	"The first paragraph gives a right that appears unconditional, but a later paragraph makes the required approval unavailable. The group needs revision before the right can be applied."
Paragraph label	Does this paragraph look clearly problematic, possibly relevant, ordinary, or uncertain?	"This paragraph is not wrong alone, but it is relevant because it provides the timing condition that controls the later obligation."
Repair packet	Which paragraphs should a reviewer see to fix the issue?	"The packet should include the grant clause, the later condition clause, and the definition paragraph. Omitting the definition would make the repair ambiguous."

Table 6: Expert-facing annotation tasks. These are the instructions used to elicit human judgments; internal roles and system outputs are used only for post-hoc analysis.

Displayed paragraph	Expert-visible text excerpt	Majority annotation
SEC EX-21 entity record A	*Entergy Louisiana Holdings LLC.* "For this consolidation path, the entity is treated as fully consolidated by the upstream registrant. The note states that no separate non-controlling interest is retained for this path."	Clear issue; included in two experts' repair packet selections.
SEC EX-21 entity record B	*ENTERGY LOUISIANA HOLDINGS INC.* "The same consolidation path reserves a 20 percent non-controlling interest for the downstream schedule. This record applies the opposite state immediately on the same path."	Clear issue; included in two experts' repair packet selections.
Case-level decision	The two records describe the same consolidation path and assign incompatible ownership states, fully consolidated with no non-controlling interest versus a retained 20% non-controlling interest.	3/3 experts mark the case as structurally conflicting; 2/3 mark the provided material sufficient, and 1/3 marks it partially sufficient.

Table 7: Representative expert-audit example. Annotators saw paragraph text and identifiers, without hidden root/witness labels or system suggestions.

Cases	Judgment	Agree	Cohen's κ
Unmodified	Existence	15/18	0.620
	Sufficiency	13/18	0.118
Injected	Existence	24/24	–
	Sufficiency	24/24	–
All	Existence	39/42	0.690
	Sufficiency	37/42	0.222

Table 8: Agreement between the first two experts before adjudication, using the updated unmodified-source annotations and retained injected-variant annotations. A dash denotes undefined κ when both experts assign the same category to every case.

with the repair reference rather than establishing that unreferenced records are irrelevant. For the 24 injected variants, we use the third expert's evidence reference on five reviewed cases; on the other 19, we average scores against the first two experts' separate references. All positive reference denominators are nonempty.

Predictions are the returned evidence core for SA-MCGS, the fixed-cap LEA output, and the direct One-shot subgraph. For SA-MCGS, we use `core_evidence_risk_subgraph_nodes`, which is available at inference time, and exclude diagnostic fields that use ground truth. Failed cases remain in the denominator. Invalid or empty outputs receive zero recall, coverage, and precision. Their outside-reference fraction is also zero by convention, so this metric must be interpreted alongside validity.

Relative to LEA, SA-MCGS improves minimal-evidence recall by 25.5 percentage points for GPT-4o (95% paired interval: 15.3–35.1) and 2.2 for Gemini 2.5 Pro (-7.4–12.0). The corresponding repair-reference precision gains are 18.6 (13.1–23.6) and 16.6 (8.5–24.4) points. For Gemini 2.5 Pro, One-shot achieves higher repair precision than SA-MCGS (91.5% versus 88.4%) but lower repair coverage (37.0% versus 68.3%), illustrating the trade-off between precise selection and repair-material coverage.

Outside-reference fractions for One-shot/LEA/SA-MCGS are 15.1/33.5/15.0% with GPT-4o and 8.5/28.2/11.6% with Gemini 2.5 Pro. Failed outputs contribute zeros to GPT-4o One-shot's value, so a low fraction alone does not imply better evidence selection. Adding the frozen injection bridge to each expert's repair set changes precision by at most 0.4 percentage points; Table 9 uses the unaugmented expert repair sets. Variant-weighted minimal-evidence recall is 38.2/61.1/88.4% for GPT-4o and 67.0/84.3/86.9% for Gemini 2.5 Pro. These are descriptive averages over the same cases, not additional independent samples.

Risk judgments on unmodified sources. Table 10 evaluates case-level risk judgments against the 15 risk and three no-risk expert labels. A recorded risk assertion requires a reasoned conflict pair with two distinct SCC node IDs; a nonempty evidence subgraph alone is not a positive judgment. The final panel reports SA-MCGS model predictions, and the archived panel retains earlier extraction records from the initial and recovery runs. Complete prediction provenance is included with the source data.

Both SA-MCGS backbones flag all 15 expert-identified risk cases. GPT-4o also correctly identifies all three no-risk cases, yielding 18/18 agreement and no false alarms; Gemini 2.5 Pro yields 15/18 agreement and three false alarms. Overall error rates are 0.0% and 16.7%, respectively. False-positive rates among the three no-risk cases are 0.0% and 100.0%. These observations support case-level risk detection on the selected unmodified sources and show that no-risk recognition depends on the backbone.

The archived runs retain their original reliability and cost accounting. One complete SA-MCGS rerun per initial failure recovers 21 of 24 failed model-cases, for 33/36 structurally valid outputs across models. Two GPT-4o LEA reruns recover one of two failures. The additional work is 63 calls and 261,389 tokens for LEA, and 1,140 calls and 5,011,988 tokens for SA-MCGS.

C Representative Long-SCC Case Study

Figure 5 compares evidence selection within the same cyclic region. One-shot retains three records but omits the root. LEA and SA-MCGS retain all risk endpoints, but SA-MCGS uses nine records compared with twelve for LEA, increasing compression from 52% to 64%. The shared layout allows direct comparison of the retained constraint and its distant witness.

D Core Algorithm Details

Algorithms 2–5 expand the overview in Algorithm 1 into detailed pseudocode. The implementation uses the same high-level objects as the main

Model	Method	Valid	Minimal recall	Repair coverage	Repair precision
GPT-4o	One-shot	20/24	40.5 [23.7, 58.3]	34.1 [21.7, 47.2]	59.9 [40.0, 78.5]
	LEA	24/24	65.4 [55.8, 75.4]	56.1 [53.3, 58.7]	66.5 [60.2, 73.0]
	SA-MCGS	24/24	**90.8** [86.8, 94.7]	**73.1** [68.7, 77.6]	**85.0** [80.9, 88.8]
Gemini Pro	One-shot	24/24	69.6 [59.9, 78.6]	37.0 [31.4, 42.9]	**91.5** [83.6, 97.7]
	LEA	24/24	86.3 [78.4, 93.6]	61.3 [55.7, 66.7]	71.8 [64.6, 79.3]
	SA-MCGS	24/24	**88.5** [81.2, 94.6]	**68.3** [59.6, 75.5]	88.4 [80.5, 95.0]

Table 9: Expert-reference evidence quality on 24 newly annotated injected variants from 14 sources. Metrics are averaged within each source, then equally across sources, and reported as percentages. Brackets give source-SCC bootstrap 95% intervals (20,000 draws, seed 20260922). Validity counts are out of 24; all planned cases remain in metric denominators. Bold/underline: best/second-best point estimate within each model (↑), not statistical significance. Shading identifies SA-MCGS.

Model	Method	Stage	TP	TN	Errors	Acc. (%)
Final SA-MCGS model predictions						
GPT-4o	**SA-MCGS**	Final	15	3	0	**100.0**
Gemini Pro	**SA-MCGS**	Final	15	0	3	83.3
Archived extraction records, rescored against the same expert labels						
GPT-4o	One-shot	Initial	11	0	7	61.1
	LEA	Initial	15	0	3	83.3
	LEA	Recovered	15	0	3	83.3
	SA-MCGS	Initial	8	0	10	44.4
	SA-MCGS	Recovered	15	0	3	83.3
Gemini Pro	One-shot	Initial	15	0	3	83.3
	LEA	Initial	14	0	4	77.8
	SA-MCGS	Initial	4	0	14	22.2
	SA-MCGS	Recovered	15	0	3	83.3

Table 10: Risk judgments on 18 unmodified sources. TP/TN denote correct risk/no-risk decisions; Errors $= 18 - \mathrm{TP} - \mathrm{TN}$. Accuracy is $(\mathrm{TP} + \mathrm{TN})/18$. Final model predictions and archived extraction records are reported separately.

paper, including a document graph G, strongly connected components S, local rollout windows W_t, relation-first memory M_t, OC node flags computed from accumulated score histories, and a final dynamic core H_T.

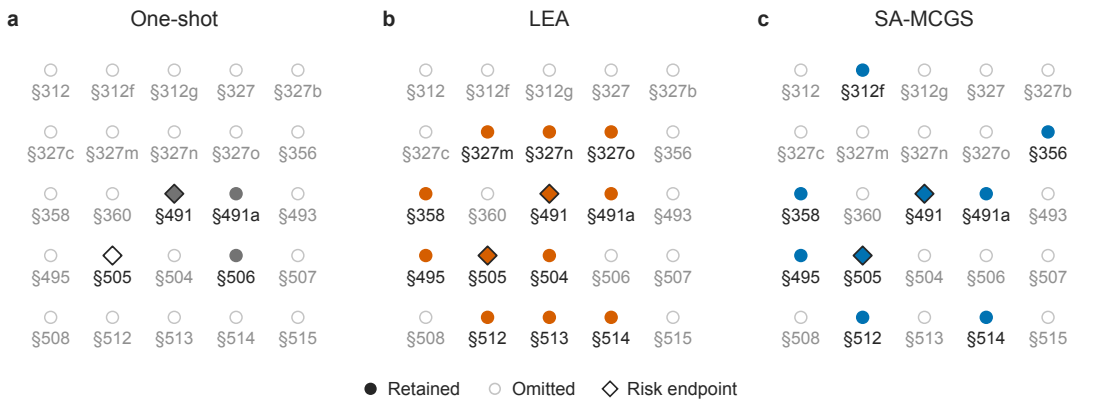
Algorithm 2 SCC-localized search

Require: Document graph $G = (V, E)$, rollout budget T, shared risk rubric
$\mathcal{S} \leftarrow$ Tarjan strongly connected components of G
Initialize collapsed graph $G' \leftarrow G$
for all component $S \in \mathcal{S}$ **do**
  if $|S| = 1$ **then**
    Keep the singleton record unchanged
  else
    Build local search unit $G_S = (S, E_S)$
    $(H_S, r_S) \leftarrow$ SA-MCGS-ROLLOUT(G_S, T) (r_S: mean-risk ranking)
    Replace S in G' with collapsed evaluation point (H_S, r_S)
  end if
end for
return Collapsed graph G' and all component risk subgraphs

Algorithm 3 SA-MCGS rollout inside one component

Require: Component $G_S = (S, E_S)$, budget T, window capacity w, shared rubric
Initialize memory M_0 with node evidence, pair ledger, and OC flags
for $t = 1$ to T **do**
  Select W_t with capacity w by frontier expansion or an eligible scheduled endpoint probe
  Reuse an eligible cached evaluation; otherwise query the LLM on W_t with the shared rubric
  Parse consumed entry $\mathcal{E}_t$: node scores, pair reports, and local subgraph
  Update node and pair memory using Algorithm 4
  Update OC flags from accumulated node-score histories
end for
Select final core H_T using Algorithm 5 on S, M_T, $\mathcal{L}_T$, and $B(S)$
return H_T and node ranking by mean observed risk $\bar{q}_T(v)$

The core is therefore not a monotone union of



	One-shot	LEA	SA-MCGS
Nodes retained	3/25	12/25	9/25
Root@3	0	1	1
Risk-any	1	1	1
Risk-all	0	1	1
Compression (%)	88	52	64

Figure 5: Comparison of three methods on the same 25-record BGB case with Gemini 2.5 Pro. Node positions are fixed across panels; filled markers indicate retained records and open markers indicate omitted records. Diamonds mark the root (§505) and witness (§491). The rows below report retained node counts and the four evaluation metrics.

Algorithm 4 Relation-first evidence memory

Require: Window W_t, consumed entry $\mathcal{E}_t$, previous memory M_{t-1}
$M_t \leftarrow M_{t-1}$
for all record $v \in W_t$ **do**
  Update average risk score and visit count for v
  Append its score to the node history and pooled scores
end for
Update unordered pair-support and explicit-report counts from $\mathcal{E}_t$ using Eq. 10
for all pair report in $\mathcal{E}_t$ (retaining endpoint order) **do**
  Store pair reasons; accumulate report-specific endpoint, AMAF, and path evidence
end for
Merge per-pair reports from $\mathcal{E}_t$ into the critical-pair ledger
Update local-subgraph and repair-entry counts from their entry-level lists, with their node and path evidence
Update window-fragment evidence and directed graph-edge visit/conflict statistics within W_t
return M_t

Algorithm 5 Dynamic core compression

Require: Component S, accumulated memory M_T and pair ledger $\mathcal{L}_T$, cap $B(S)$
Construct C_T using Eq. 13; compute $s_T(v)$ using Eq. 7
Order positive-priority candidates by decreasing s_T, then decreasing evidence-channel and qualifying-pair counts, then SCC order; scale adjacent-priority gaps by the highest priority
if the order is empty **then**
  return First node of C_T in SCC order, or $\emptyset$ if $C_T = \emptyset$
end if
Select the configured protected-pair shortlist; let P_T contain its endpoints
Initialize $H_T = \emptyset$; seed protected pairs within $B(S)$, then strong pairs within the seed limit
for all unselected v in the fixed priority order **do**
  Stop if $|H_T| = B(S)$
  Once the minimum core size is reached, stop on absolute/relative-score or adjacent-gap thresholds unless $v \in O_T \cup P_T$
  $H_T \leftarrow H_T \cup \{v\}$
end for
Add supported counterparts satisfying the pair-closure rule within $B(S)$
return H_T in SCC order

all suspicious records. Evidence from later rollouts can change the final selection, with protected-pair seeding prioritizing repeatedly supported endpoints under the cap.

E Formula Details for Evidence Updates

Implementation-level equations summarized in Section 3 are expanded here. The main text retains the search template plus two scoring components, the priority score and core objective. Remaining details were fixed before evaluation and left untuned on test cases.

Window selection and evaluation reuse. Window capacity w is independent of graph radius; the reported local-window experiments use $w = 4$. On an eligible scheduled probe, the selector first considers a critical-pair revisit, then a supported relation pair. Scheduling uses configured warm-up and periodic gates; critical revisits also obey their

own gate. If no probe qualifies, the selector chooses a UCB seed and expands the incoming/outgoing adjacency frontier under Eq. 4. An empty frontier permits at most one randomly chosen remaining node before stopping. A probe includes its endpoints and a shortest path in the undirected projection when the path fits the capacity; otherwise it starts from the endpoints. Remaining positions are filled from neighbors ranked by edge and node priors, then by component-wide node evidence, always respecting w. Windows need not be full or connected, and displayed graph edges retain their original directions.

Cache lookup uses the unordered window-node set. An existing entry is reused while its reuse counter is below `tt_max_reuse`; otherwise the evaluator is invoked. Reused entries still update memory. The search counter `llm_calls` counts returned non-cache evaluations, including caught failures that yield default scores, rather than underlying API attempts. Retries and restarted executions can add requests, so recorded call/token usage follows the logs and recovery accounting in Appendix K.9. Each L_j includes the serialized record text, visible edges, and prompt template; a fixed record count does not fix token length. Beyond scalar node/graph-edge statistics, cached outputs, accumulated score histories, observed-pair records, and retained traces occupy storage that grows with the saved observations. The returned-core cap does not bound this storage.

Exploration-score implementation. Algorithms and schematics use serial notation; concurrent workers select from launch-time snapshots and update memory in completion order. All terms below refer to the selection-time snapshot. Let $A(x)$ denote the first three terms of Eq. 4. The implementation mixes this score with normalized Gamma noise as $(1-d)A(x) + dg\xi(x)$. For $0 \le d < 1$, dividing finite candidate scores by the common positive factor $1-d$ preserves the selected maximizer and yields

$$\eta(x) = \frac{dg}{1-d}\xi(x).$$

Gamma draws use the configured shape and unit scale, divided by their sum plus 10^{-8}. The scale is $g = 10$ for seeds and $g = 5$ for frontier edges. This mapping preserves selection, not the numerical mixed scores. For seeds, N is the total node visit count; for frontier expansion it is the sum over the current candidate edges, both floored at one. The term ℓ includes the configured virtual-loss weight. A seed with $n + \ell = 0$ receives infinite priority; a frontier edge uses that branch only if its statistics are absent. Multiple infinite scores are resolved by candidate order. Progressive bias uses actual visits, with $\lambda_p =$ `relation_prior_weight` and $\delta =$ `relation_prior_decay`; the exploration denominator uses $\epsilon = 10^{-8}$.

Core priority. Equation 7 uses $\alpha = 0.24$ and $\gamma = 0.16$. Each nonnegative evidence channel is divided by its maximum within the SCC. Channels whose maximum exceeds 10^{-12} form a shared averaging set for all nodes; missing entries contribute zero, and $e_T(v) = 0$ if this set is empty. The channels measure risk above the median over visited nodes, conflict rate, local-subgraph appearances, repair-entry support, conflict-pair endpoint counts, OC flags, AMAF evidence, path-fragment evidence, relation-prior support, and critical-pair endpoint support. The last channel sums retained critical-pair scores incident to the node, multiplying locked entries by 1.25.

For a locally supported pair $p = \{u, v\}$, the initial score is

$$r_T^0(p) = 0.45\sigma(p) + 0.20\widehat{x}_T(p) + 0.20\widehat{m}_T(p) + 0.15[e_T(u) + e_T(v)]/2,$$

Here σ is semantic conflict strength; hats denote log-normalized explicit-report counts $x_T(p)$ and pair support $m_T(p)$. Each count is transformed by $\log(1 + \text{count})$ and divided by the maximum transformed count of its type across local pairs, with zero used when that maximum is zero. A retained critical-pair entry ($\kappa_T(p) \ge \tau_{\text{exit}}$) sets $r_T(p) = \max\{r_T^0(p), \kappa_T(p)\}$, taking a missing local score as zero; otherwise $r_T = r_T^0$.

Let $\mathcal{Q}_T$ contain merged observed pairs with both endpoints in S satisfying at least one condition: $r_T(p) \ge \tau_{\text{pair}}$, semantic strength at least τ_{sem}, or a positive explicit-report count. Merged records use the maximum local and retained critical-pair semantic strength. For Eq. 7, $\mathcal{R}_T(v)$ contains incident pairs in $\mathcal{Q}_T$ and incident protected pairs. Protection eligibility means a locked ledger state or $\kappa_T(p) \ge \tau_{\text{core}}$ with at least L supports. Defaults are $\tau_{\text{pair}} = 0.42$, $\tau_{\text{sem}} = 0.65$, $\tau_{\text{exit}} = 0.48$, $\tau_{\text{core}} = 0.62$, and $L = 2$. O_T is the persistent set of OC-flagged nodes.

Node and relation updates. For every record in the consumed window W_t, node risk evidence is

updated by a visit-counted running average:

$$n_t(v) = n_{t-1}(v) + 1,$$
$$\bar{q}_t(v) = \frac{n_{t-1}(v)\bar{q}_{t-1}(v) + y_t(v)}{n_t(v)}. \tag{9}$$

Index t counts consumed entries $\mathcal{E}_t$ in completion order, including cache reuse; missing node scores default to 0.5. For an unordered pair $p = \{u, v\}$ of distinct SCC nodes, let $\mathcal{U}_t$ be the set of valid pairs appearing in the entry's `conflicts` or `local_conflict_edges`. Let $k_t^{\text{exp}}(p)$ count valid reports for p in `conflicts`. Local support $m_t(p)$ and explicit-report count $x_t(p)$ start at zero and accumulate as

$$m_t(p) = m_{t-1}(p) + \mathbf{1}[p \in \mathcal{U}_t],$$
$$x_t(p) = x_{t-1}(p) + k_t^{\text{exp}}(p). \tag{10}$$

Thus local support increases at most once per pair per consumed entry, whereas explicit counts follow report multiplicity; a local edge alone is not an explicit report. Relation reasons are retained separately. Reports for each pair are merged before one critical-ledger update per entry, retaining their explicit/local report counts. These counts describe consumed evidence, not independent model calls.

Endpoint and directional AMAF/path increments remain report-specific; deduplicating pair support does not deduplicate those increments. Local-subgraph and repair-entry counts are updated from the entry's node lists outside the relation loop. Window-fragment evidence uses the ordered window and the entry's context nodes. Directed graph edges with both endpoints in the window receive separate visits; their conflict indicator is one when both endpoints occur in the entry's explicit-conflict node set. Algorithm 4 summarizes these updates; Algorithms 1 and 3 then compute OC flags from the accumulated node histories.

Relation priors and critical pairs. Equation 5 summarizes base normalization. Nodes use $\mathbf{w} = \theta$ with repair-entry support, local-subgraph appearance, pair-endpoint support, and OC node flags. Edges use $\mathbf{w} = \beta$ with critical-pair score, local semantic strength, explicit reports, pair support, directional AMAF, and accumulated conflict evidence. Time-indexed priors use completed-evidence snapshot M_t; Eq. 4 reads the current selection-time snapshot. For exact normalization, let a_t^V and a_t^E be the nonnegative weighted node and edge scores, and M_t^V, M_t^E their maxima. The node maximum ranges over S; the edge maximum ranges over the original directed edges and their reverse keys. The implementation returns zero when the relevant maximum is at most $\epsilon_\pi = 10^{-12}$; otherwise it divides by that maximum without the additive ϵ in the compact template. Thus node prior $\pi_t(v) = a_t^V(v)/M_t^V$ when $M_t^V > \epsilon_\pi$. For a frontier edge e and its endpoint v_{new} outside the current window W, the positive-maximum branch is

$$b_t^E(e) = a_t^E(e)/M_t^E, \quad M_t^E > \epsilon_\pi,$$
$$\pi_t(e; W) = \min\{1, b_t^E(e) + \beta_{\text{enter}}\pi_t(v_{\text{new}})\}. \tag{1327}$$

Here $\beta_{\text{enter}} = 0.25$; no node bonus is added in the zero-maximum branch. Disabling relation-first selection sets both priors to zero. In Table 11, $\beta_{\text{critical}} = 1.55$ multiplies the full critical-pair score, without final-core eligibility filtering, and $\beta_{\text{semantic}} = 1.35$ multiplies semantic strength from local pair reasons. The reverse-AMAF feature is first scaled by 0.55, then multiplied by $\beta_{\text{reverse}} = 0.35$. A separate bounded ledger records persistent critical-pair hypotheses:

$$\kappa_t(u, v) = \min\{1, \omega^\top \chi_t(u, v)\}, \tag{11}$$

for $\{u, v\} \in \mathcal{L}_t$, the unordered node pairs recorded in the ledger after consuming entry $\mathcal{E}_t$. Here $\chi_t(u, v)$ contains pair-level evidence such as semantic incompatibility, severity, same-path support, no-fallback language, repeated support, and explicit conflict reports. The support and explicit-report features use clipped log-normalized ledger counts:

$$\chi_{\text{support}} = \min\left(1, \frac{\log(1+n)}{\log 7}\right),$$
$$\chi_{\text{explicit}} = \min\left(1, \frac{\log(1+x)}{\log 5}\right), \tag{1347}$$

where n and x count support and explicit reports for that ledger entry.

Online conflict signal. Consistent with the memory updates, t indexes consumed entries in completion order; i_t is the triggering rollout's zero-based launch index. Let $\mathcal{Y}_t(v)$ be the multiset of risk scores backpropagated to node v, and $\mathcal{P}_t = \biguplus_{v \in S} \mathcal{Y}_t(v)$ the pooled multiset within the current SCC. Both include the triggering update and scores from cached evaluations. Let μ_t and σ_t be the pooled mean and population standard deviation. The pooled median $\tilde{q}_t$ is the element at one-based

position $\lfloor |\mathcal{P}_t|/2 \rfloor + 1$ after sorting, using the upper middle value for an even-sized pool.

The shared parameter $w_{\mathrm{OC}} =$ `detection.min_rollouts` controls rollout warm-up and minimum node observations; its implementation default is 8. If $|\mathcal{P}_t| \geq 10$, $i_t \geq w_{\mathrm{OC}}$, and $\sigma_t \geq 10^{-6}$, the eligible set is $\mathcal{A}_t = \{v \in S \setminus O_{t-1} : |\mathcal{Y}_t(v)| \geq w_{\mathrm{OC}}\}$; otherwise $\mathcal{A}_t = \varnothing$. For eligible nodes, write $\bar{q}_t(v) = \mathrm{mean}(\mathcal{Y}_t(v))$ and define

$$Z_t(v) = \frac{\bar{q}_t(v) - \mu_t}{\sigma_t},$$
$$F_t(v) = \frac{1}{|\mathcal{Y}_t(v)|} \sum_{a \in \mathcal{Y}_t(v)} \frac{\#\{b \in \mathcal{P}_t : b \leq a\}}{|\mathcal{P}_t|},$$
$$D_t(v) = \frac{1}{|\mathcal{Y}_t(v)|} \sum_{a \in \mathcal{Y}_t(v)} \mathbf{1}[a > \tilde{q}_t].$$

Here $F_t(v)$ is the average empirical percentile, and $D_t(v)$ is the fraction above the pooled median. With $O_0 = \varnothing$, flags accumulate as

$$\begin{aligned} g_t(v) &= \mathbf{1}[Z_t(v) > \tau_z] + \mathbf{1}[F_t(v) > \tau_f] \\ &\quad + \mathbf{1}[D_t(v) > \tau_d], \\ O_t &= O_{t-1} \cup \{v \in \mathcal{A}_t : g_t(v) \geq 2\}. \end{aligned} \tag{12}$$

The thresholds are $\tau_z = 1.5$ and $\tau_f = \tau_d = 0.8$, with strict inequalities for all three votes. Previously flagged nodes remain flagged until the SCC search ends. Flagged nodes remain in OC memory and receive the bounded bonus in Eq. 7.

Core candidate pool. In Section 3.6, $V_T^{\mathrm{ev}} = \{v \in S : e_T(v) > 0\}$ and $\mathrm{End}(\mathcal{Q}_T) = \bigcup_{p \in \mathcal{Q}_T} (p \cap S)$. The final candidate pool in Eq. 8 is

$$C_T = \begin{cases} C_T^{\circ}, & C_T^{\circ} \neq \varnothing, \\ C_T^{\mathrm{ctx}}, & C_T^{\circ} = \varnothing, \end{cases} \tag{13}$$

where C_T^{ctx} is the provisional context set constructed before core selection. If that set is also empty in a nonempty SCC, it is initialized with the node of highest mean observed risk, breaking ties by SCC order.

F Fixed Search-Prior Coefficients

The numeric values in Table 11 are fixed search-prior coefficients, not learned parameters. They were selected using pilot checks before the locked evaluation. Domain experts were used only as a sanity check on the ordinal reliability ordering among evidence types, not to tune coefficients on reported test cases. Prior MCTS work motivates the structure, including UCB/UCT exploration (Kocsis and Szepesvári, 2006), visit-decayed heuristic priors through progressive bias (Chaslot et al., 2008; Browne et al., 2012), AMAF/RAVE-style evidence reuse (Gelly and Silver, 2007), and graph/transposition-aware value reuse (Childs et al., 2008; Czech et al., 2021). The coefficients instantiate these ideas for structural document risk.

G Shared Risk Rubric Prompt

Reviewer fairness depends on separating the risk definition from the search mechanism. The following excerpt gives the shared evaluator rubric used by one-shot prompting and the local-window evaluators used by LEA and SA-MCGS.

> You are given records from a directed graph of interdependent documents. Identify records or record sets that are structurally inconsistent, mutually incompatible, underspecified, or high-impact if left unresolved. Use only visible record text and visible directed relations. Do not assume a domain-specific defect type. Return evidence-based judgments with node ids and concise reasons.

Shared semantics. One-shot prompting, LEA, and SA-MCGS share this risk rubric. One-shot produces a global ranking and subgraph in one call; LEA aggregates fixed local evaluations; SA-MCGS adaptively acquires local evidence to construct its final core.

H Prompt Templates

Complete prompts include serialization details and JSON schemas. The abstract templates reported here are sufficient to reproduce the evaluation interface without depending on one implementation file.

One-shot prompting template.

> Input: the complete node list for one strongly connected component, the text of every node, and all directed edges among the nodes. Apply the shared risk rubric to the full component. Return valid JSON with `global_ranking`, `risk_subgraph_nodes`, `risk_factor_distribution`, and `explanation`. Rankings and subgraph selections should be grounded in node IDs and visible edges.

SA-MCGS local-window prompt.

> Input: a local window from one strongly connected component, including visible node text, visible directed edges, and a concise prior

Parameter	Value	Role and rationale
Node-feature weights θ **for Eq. 5**		
θ_{repair}	1.10	LLM names the record as a repair entry. This is closest to the final remediation target.
θ_{local}	0.90	The record appears in local risk subgraphs; useful but noisier than explicit repair-entry evidence.
θ_{pair}	0.85	The record is an endpoint of a supported conflict pair; structural risks often require two endpoints.
θ_{node}	0.70	AMAF/RAVE-style node evidence reused across rollouts; important but indirect.
θ_{path}	0.75	Path-fragment evidence that the record lies on repeatedly implicated local traversals.
θ_{conflict}	0.60	Single-window conflict reports; useful but downweighted because local LLM judgments can be noisy.
θ_{OC}	2.00	Node OC flag computed from score histories; routing bonus, normalized and visit-decayed.
Edge-feature weights β **for Eq. 5**		
β_{critical}	1.55	Critical-pair score from the accumulated ledger.
β_{semantic}	1.35	Semantic strength derived from local pair reasons.
β_{explicit}	0.90	Explicit conflict wording helps, but is downweighted relative to persistent structural evidence.
β_{pair}	0.70	Repeated pair support indicates relation-level consistency across windows.
β_{edge}	0.65	AMAF edge evidence reused from related traversals.
β_{reverse}	0.35	Reverse AMAF evidence, first scaled by 0.55, provides auxiliary support.
β_{conflict}	0.55	The edge's own accumulated conflict estimate, kept below critical-pair score and semantic strength.
Critical-pair prior ω **in Eq. 11**		
ω_{sem}	0.30	Semantic incompatibility dominates because it directly captures mutual exclusion.
ω_{sev}	0.25	Severity is second: critical risks deserve revisits, but severity alone can be diffuse.
ω_{path}	0.18	Same-path evidence strengthens the claim that two endpoints jointly define one repair region.
$\omega_{\text{nofallback}}$	0.12	No-fallback language makes contradictions harder to resolve locally.
ω_{support}	0.10	Repeated support stabilizes initially uncertain pair hypotheses.
ω_{explicit}	0.05	Explicit conflict reports are useful but lowest-weighted to avoid rewarding one noisy sentence.

Table 11: Fixed search-prior coefficients used by SA-MCGS. Higher values reflect more reliable or more directly repair-relevant evidence. Values were frozen before the locked evaluation and are not learned from the reported test cases.

> evidence summary. Apply the same shared risk rubric to the local window. Return valid JSON with `local_node_scores`, `relation_evidence`, `local_risk_subgraph_nodes`, `repair_entry_nodes`, and a concise explanation grounded in exact node IDs.

Output contrast. One-shot prompting ranks the entire component and returns a final subgraph in one generation. LEA and SA-MCGS share a local evaluator interface and aggregate its outputs algorithmically. LEA uses fixed windows without adaptive search or memory; Appendix K.1 specifies its window construction and aggregation.

I Dataset and Injection Details

Data sources and attribution. We use CUAD, released by The Atticus Project under CC BY 4.0 (Hendrycks et al., 2021). We use CARDS dependency graphs by Euxane Tran-Girard (ODbL 1.0), together with package records from Debian. We use CorpWatch SEC EX-21 records distributed by OpenSanctions under CC BY-NC 4.0. We use German BGB text from Gesetze im Internet, exempt from copyright protection under Section 5(1) of the German Copyright Act.

Formatting interface. Record formats vary by domain: Debian records contain package fields,

Domain	Why selected	Graph construction	Role in evaluation
Debian	Technical dependency graph with explicit package relationships.	Package metadata entries become records; dependency, replacement, conflict, and virtual-provider fields become directed edges.	Evaluates software migration and dependency constraints beyond legal and financial text.
SEC EX-21	Public company subsidiary disclosures with entity-control structure.	Subsidiary names, jurisdictions, consolidation fields, ownership hints, and control relations become entity records and directed edges.	Evaluates ownership and consolidation reasoning from public financial and legal disclosures.
BGB	Official German Civil Code cross-references.	Statutory sections become records; references, obligations, conditions, exceptions, and remedies become directed edges.	Provides a relatively clean setting for statutory cross-reference cycles in authoritative legal text.
CUAD	Expert-labeled commercial contract dataset.	Clauses and definitions become records; definition use, cross-reference, amendment, condition, termination, and obligation relations become edges.	Tests long contract components with ambiguity and diffuse evidence.

Table 12: Domain coverage, dataset construction, and evaluation roles. All domains are represented as directed record graphs before SCC localization.

Template	Root-side edit	Witness / affected edit	Intended risk
`direct_mutex`	Adds a direct obligation, ownership, compatibility, or condition statement expected to hold.	Adds a distant statement that requires the opposite state under the same dependency chain.	Two records are individually plausible but mutually incompatible when the component context is followed.
`handoff_invariant`	States that an invariant is expected to be preserved before control, migration, or responsibility is handed off.	States that the downstream record can proceed only after the invariant has been changed or removed.	The handoff cannot be completed because the precondition and downstream entry condition disagree.
`temporal_gate`	Specifies a deadline, waiting period, migration window, vesting condition, or trigger order.	Requires a conflicting temporal gate, such as earlier execution, delayed activation, or a missing prerequisite.	The same chain imposes incompatible timing constraints.
`condition_trigger`	Makes a right, remedy, interface, or ownership statement conditional on a trigger.	Makes the same trigger impossible, optional, or reversed for a dependent record.	A dependent record relies on a condition whose truth value is changed elsewhere in the component.

Table 13: Critical structural injection templates. All templates edit existing records only and avoid explicit defect labels.

BGB records are statutory sections, SEC EX-21 records are subsidiary disclosures, and CUAD records are contract clauses. Lightweight domain formatters

$$\mathcal{A}_d = (\text{serialize}_d, \text{edges}_d, \text{normalize}_d) \quad (14)$$

serialize records, expose typed directed edges, and normalize model outputs without specifying a target risk type or revealing injected endpoints. All three methods apply the same domain-neutral rubric to visible node text and directed edges, identifying structural inconsistency, mutual incompatibility, underspecification, and high-impact risks. They differ in how they acquire and aggregate evidence: global generation, fixed local aggregation, or adaptive SCC-aware search.

Table 12 summarizes the four included domains and their conversion into directed record graphs. Domain coverage spans several forms of authority and structure, including technical dependency metadata, public corporate disclosures, official statutory references, and expert-labeled contract clauses.

SCC construction and filtering. For each domain, we construct a directed record graph and enumerate non-trivial SCCs using Tarjan's algorithm. Candidate components are filtered by size and record availability and then locked before model evaluation. The main experiment uses the same locked SCC blocks for One-shot, LEA, and SA-MCGS under the same model and profile.

Injection constraints. The critical benchmark modifies only existing records in the selected SCC. No new nodes are added. The injected root record provides the endpoint used for Root@3 evaluation. A distant witness record supplies the evidence that makes the root-side statement structurally incompatible under the graph context. Affected records are downstream or neighboring records whose interpretation changes under the root–witness conflict. Subgraph metrics count root, witness, and affected records as repair-relevant endpoints when applicable. Table 13 lists the four injection templates.

Why Risk-all matters. Risk-any is useful when a repair process can sometimes start from one good entry point. Risk-all is stricter and central for critical cases, where fixing a structural contradiction often requires both sides of the incompatibility. A subgraph that includes only the obvious-risk paragraph but omits the distant condition may be too small to support a reliable repair.

J Additional Experimental Details

Locked run configuration. All five models are evaluated on the same 80 injected variants from 20 source SCCs. One-shot, LEA, and SA-MCGS use the shared risk rubric and fixed method configurations specified in Appendix K.1. Table 14 groups results by model, domain, and SCC size.

Paired Risk-all outcomes favor SA-MCGS over LEA for GPT-4o, Qwen2.5-72B, and Llama 3.1-8B-Instruct (Figure 6b). For GPT-4o, 32 cases succeed only with SA-MCGS, compared with 7 only with LEA.

J.1 Performance by SCC size

SA-MCGS improves Risk-all over LEA in all three SCC-size groups for GPT-4o and Qwen2.5-72B (Figure 7). Their largest gains occur in medium components: 37.50 and 62.50 percentage points, respectively. For Llama 3.1-8B-Instruct, Root@3 also improves in every size group. Absolute results appear in Figure 8 and Table 14.

Figure 8 uses the same size groups and case counts as Figure 7, reporting Root@3 and Risk-all for all three methods and five models.

J.2 Budget-prefix convergence

Most gains in evidence completeness occur within the first 20 rollouts (Figure 9): Risk-all rises from 25.00% at budget 5 to 76.25% at budget 20. Subsequent rollouts refine the core, reaching 75.00% Risk-all and 51.26% compression at budget 60, matching the GPT-4o results in Table 2.

K Additional Baseline, Ablation, and Cost Experiments

K.1 Local Evidence Aggregation Protocol

Local Evidence Aggregation (LEA) provides a structured baseline based on local graph retrieval and evidence aggregation, without implementing a full external GraphRAG system. For each SCC node, LEA constructs a four-node window from the node and its incoming and outgoing neighbors, using the fixed SCC node order. Missing positions are filled by alternating forward and backward offsets in that order. Each window is evaluated with the same local prompt builder, response schema, and parser as SA-MCGS, producing node scores, local risk nodes, conflict relations, critical pairs, and repair entries. Repeated evaluations may use cached results. The number of windows depends on graph size rather than a fixed rollout budget.

Let $\bar{q}_v$ denote the mean risk score over windows containing node v, and let L_v, R_v, and E_v count its occurrences in local risk sets, repair entries, and conflict-edge endpoints, respectively. LEA ranks roots by $\bar{q}_v$ and assigns each node a subgraph membership score

$$a_v = L_v + R_v + E_v + \max\{0, \bar{q}_v - \mathrm{median}_u\, \bar{q}_u\}. \tag{15}$$

LEA returns the top $k = \min\{|S|, \max(4, \lceil 0.45|S| \rceil)\}$ nodes, breaking ties by SCC order. If all membership scores are zero, it selects nodes by mean risk score. LEA uses no adaptive relation-memory selection, critical-pair revisiting, OC-guided core construction, or dynamic replacement. It therefore shares the local evidence evaluator with SA-MCGS but uses a simpler aggregation procedure. In this original five-model comparison, computational cost is reported separately and token budgets are not matched. Appendix K.2 adds repeated-sampling baselines with per-case SA-referenced budgets.

The main comparison comprises 80 injected variants from 20 source SCCs across Debian, SEC, BGB, and CUAD. GPT-4o, Gemini 2.5 Pro, Qwen2.5-72B, Llama 3.1-8B-Instruct, and Qwen3.5-9B each run One-shot, LEA, and SA-MCGS on the same inputs, yielding 400 model-cases and 1,200 method outputs. Inputs are matched across methods by source content, SCC

A. Models

Group	Method	N	Root@3	Risk-any	Risk-all	Comp.
GPT-4o	One-shot	80	26.25	55.00	15.00	**67.89**
	LEA	80	78.75	96.25	43.75	51.26
	SA-MCGS	80	**86.25**	**100.00**	**75.00**	51.26
Gemini 2.5 Pro	One-shot	80	**90.00**	97.50	**85.00**	**74.98**
	LEA	80	87.50	**100.00**	83.75	51.26
	SA-MCGS	80	87.50	97.50	83.75	55.86
Qwen2.5-72B	One-shot	80	2.50	46.25	13.75	42.69
	LEA	80	67.50	**97.50**	43.75	51.26
	SA-MCGS	80	**70.00**	**97.50**	**75.00**	**52.82**
Llama 3.1-8B-Instruct	One-shot	80	3.75	47.50	25.00	30.27
	LEA	80	43.75	92.50	35.00	51.26
	SA-MCGS	80	**61.25**	**95.00**	**48.75**	**51.30**
Qwen3.5-9B	One-shot	80	12.50	53.75	12.50	62.70
	LEA	80	56.25	**91.25**	**63.75**	51.26
	SA-MCGS	80	**62.50**	86.25	**63.75**	**67.25**

B. Domains

Group	Method	N	Root@3	Risk-any	Risk-all	Comp.
Debian	One-shot	40	25.00	95.00	52.50	46.76
	LEA	40	65.00	**100.00**	**85.00**	52.27
	SA-MCGS	40	**80.00**	**100.00**	80.00	**55.44**
SEC EX-21	One-shot	100	35.00	89.00	46.00	48.82
	LEA	100	**96.00**	97.00	60.00	49.80
	SA-MCGS	100	94.00	**100.00**	**76.00**	**50.58**
BGB	One-shot	60	21.67	63.33	23.33	57.25
	LEA	60	46.67	85.00	26.67	50.33
	SA-MCGS	60	**51.67**	**90.00**	**51.67**	**57.42**
CUAD	One-shot	200	25.00	37.50	20.00	**76.05**
	LEA	200	58.50	**97.00**	53.00	52.06
	SA-MCGS	200	**68.50**	93.50	**69.00**	57.85

C. SCC sizes

Group	Method	N	Root@3	Risk-any	Risk-all	Comp.
Short ($\leq$12)	One-shot	180	35.00	83.33	42.78	49.79
	LEA	180	77.22	94.44	67.22	50.28
	SA-MCGS	180	**82.78**	**98.33**	**73.33**	**52.41**
Mid (14–20)	One-shot	120	18.33	47.50	20.83	**68.94**
	LEA	120	57.50	**97.50**	40.83	51.32
	SA-MCGS	120	**73.33**	**97.50**	**72.50**	55.05
Long ($\geq$24)	One-shot	100	23.00	33.00	19.00	**82.82**
	LEA	100	**59.00**	**95.00**	46.00	52.94
	SA-MCGS	100	57.00	87.00	**58.00**	62.42

Table 14: Five-model results by model, domain, and original SCC size. Metrics are percentages; domain and size summaries pool the same five models. Bold/underline mark highest/second-highest distinct values within each group, including ties. Higher compression means a smaller core. Shading identifies SA-MCGS.

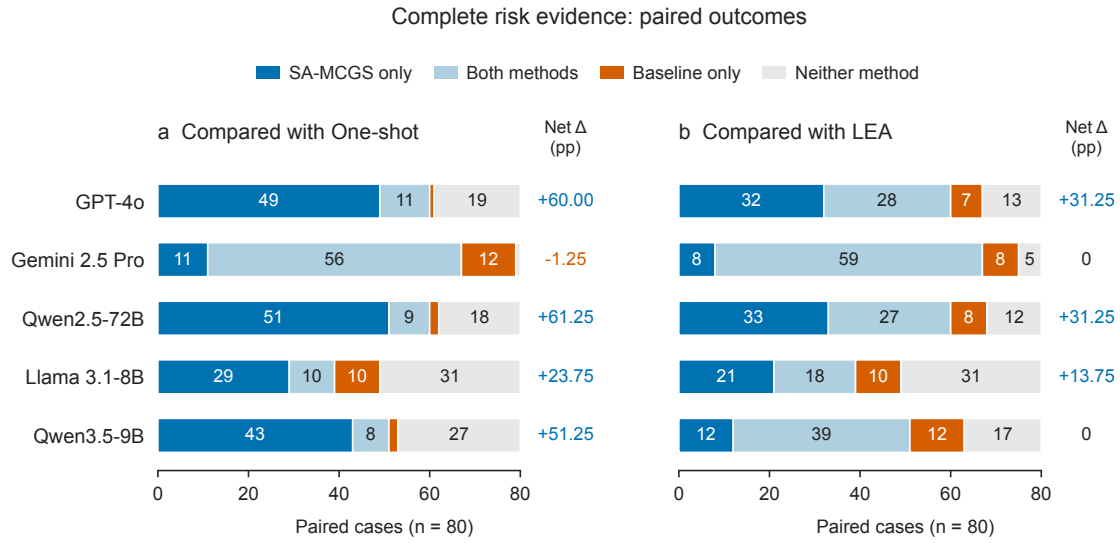


Figure 6: Paired Risk-all outcomes for SA-MCGS versus (a) One-shot and (b) LEA on the same 80 cases per model. Bars separate cases where complete evidence is retained by SA-MCGS only (blue), both methods (light blue), the baseline only (orange), or neither method (gray). Net Δ is the Risk-all difference between SA-MCGS and the baseline, in percentage points.

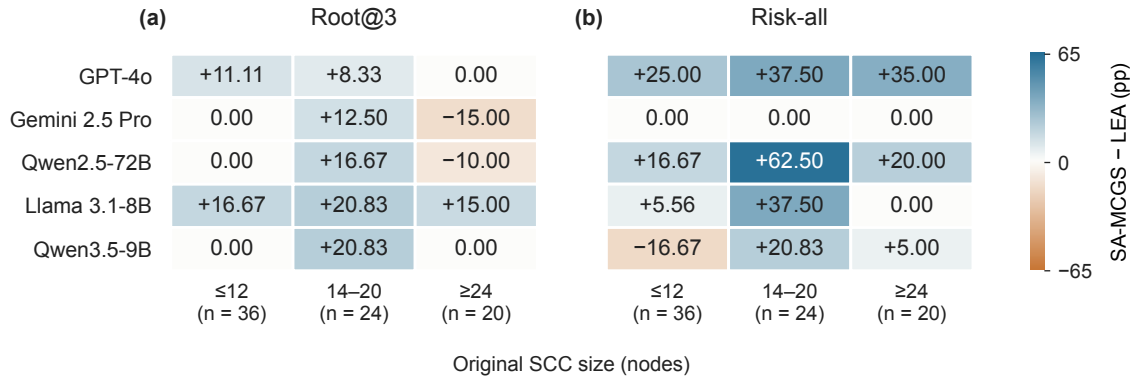


Figure 7: SA-MCGS gains over LEA by SCC size: (a) Root@3 and (b) Risk-all. Cells show percentage-point differences for each model and size group (36 small, 24 medium, and 20 large cases per model). Blue indicates gains and orange indicates losses on a shared zero-centered scale. Figure 8 reports absolute performance for all three methods.

membership, and injection identity. All Qwen3.5-9B runs use `enable_thinking=false`. The separate contemporaneous cost experiment uses 12 source SCCs and 72 method jobs.

Oracle reference. The historical oracle-risk Top-3-from-10 reference uses ground-truth metrics to select among ten full-SCC attempts. For each three-attempt subset of valid outputs, it computes mean Root@3, Risk-any, and Risk-all. Selection lexicographically maximizes the minimum of these three metrics, their mean, and mean compression. With fewer than three valid attempts, it uses all valid attempts; with none, binary metrics are zero. Reported metrics average the selected attempts without pooling their nodes or subgraphs. This oracle selection requires ground truth, unlike deployable One-shot and LEA, and incurs the computation of ten attempts.

K.2 Repeated-Sampling Baselines with SA-Referenced Budgets

Cases and budget. This comparison uses the same 80 injected variants from 20 source SCCs, with four variants per source: 40 CUAD, 20 SEC, 12 BGB, and 8 Debian cases. We evaluate GPT-4o, Qwen2.5-72B, Llama 3.1-8B-Instruct, Qwen3.5-9B, and Kimi-VL-A3B. Historical SA-MCGS outputs remain fixed. For model m and case i, LEA and Full-SCC are assigned the same reference bud-

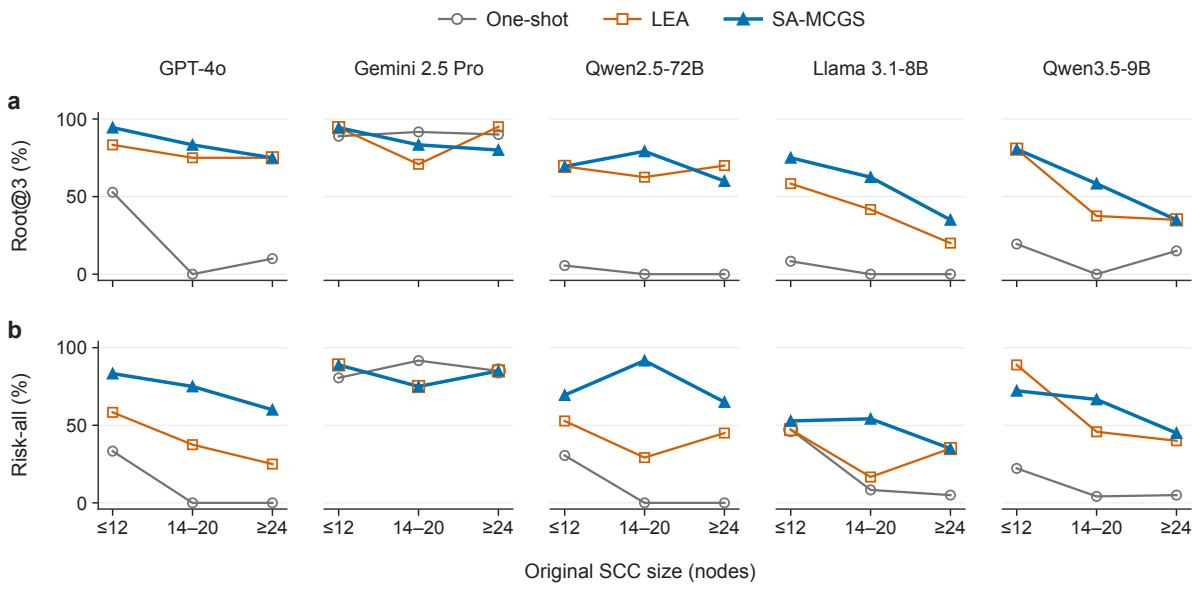


Figure 8: Performance by SCC size. Columns correspond to models and rows to (a) Root@3 and (b) Risk-all. All three methods use the same small ($\leq$ 12 nodes), medium (14–20), and large ($\geq$ 24) case groups. Gray circles, orange squares, and blue triangles denote One-shot, LEA, and SA-MCGS, respectively.

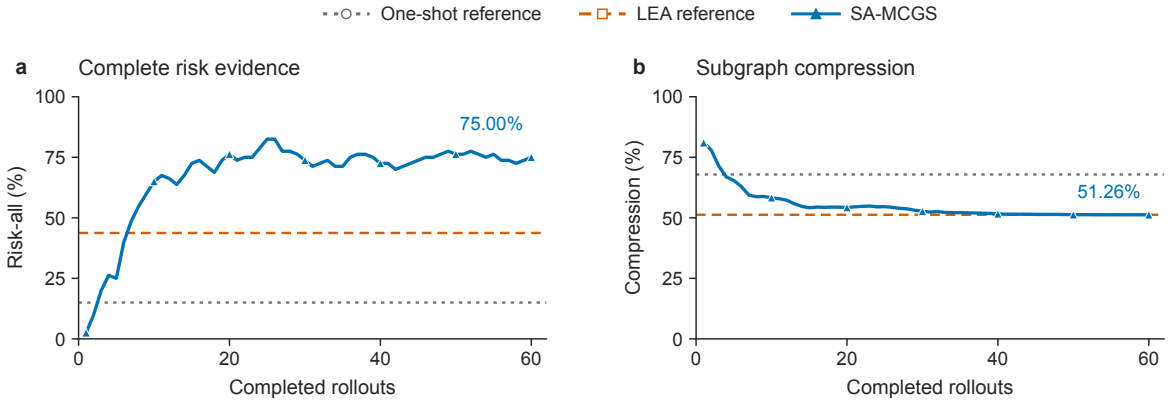


Figure 9: GPT-4o evidence retention and compression across 60 rollout prefixes on the same 80 cases. The solid curve represents SA-MCGS; horizontal lines indicate the final One-shot and LEA scores. Each prefix is reconstructed from stored search evidence using the final-core selection rule. Results at budget 60 match the final outputs.

get $B_{m,i}$ for candidate generation and selection. The reference uses recorded historical SA-MCGS token consumption where available, or calibration from existing usage records: same-model, same-domain SA cost measurements for GPT-4o, and historical SA call counts combined with same-case LEA tokens per call for Qwen2.5-72B.

Repeated generation and selection. Each LEA candidate reruns the complete four-node-window aggregation procedure in Appendix K.1; each Full-SCC candidate reanalyzes the entire component. Both retain their native root ranking and output core. The reference budget and selector context capacity determine the candidate allocation; the number of candidates is not fixed. With one candidate, it is returned directly. With multiple candidates, the same model acts once as a judge, reading the source records, graph relations, and anonymized candidates, including their rankings, cores, and up to 16 reported conflict pairs. It selects one existing candidate without merging or editing outputs. Candidate order is shuffled with a fixed seed, and selection never accesses injection labels or evaluation metrics.

Inference configuration. Candidate sampling uses temperature 0.4 and the judge uses tempera-

ture 0. Local-window, Full-SCC, and judge output limits are 2,048, 12,000, and 256 tokens, respectively, with request allowances adjusted to the reference budget. Qwen3.5-9B thinking is disabled, and the local vLLM context limit is 131,072 tokens. Relative to the original Full-SCC one-shot run, this protocol enables stochastic candidate sampling.

Coverage and compactness. Table 15 compares all five backbones. On GPT-4o, SA-MCGS reaches 86.25% Root@3, versus 71.25% for LEA and 22.50% for Full-SCC. On Qwen2.5-72B, it reaches 70.00%, versus 55.00% and 23.75%, respectively, and its 52.82% compression exceeds both baselines. For Qwen3.5-9B, Full-SCC achieves the highest compression (71.76%) but only 12.08% Risk-all. Kimi shows the reverse balance: Full-SCC achieves the highest Risk-all but only 13.60% compression. These results show that the coverage–compression balance depends on the backbone.

K.3 Stage-1 Component Ablations

Table 16 compares Full, no relation-first memory, no critical-pair ledger/revisit, and random local windows on 80 variants from 20 source SCCs. Removing relation-first memory disabled relation-biased selection and relation probes; removing the critical-pair ledger disabled its protection and revisiting; random-window selection replaced the local-window selector. The table measures endpoint coverage in the returned core separately from root-ranking performance.

Full attained 77.50% Risk-all, compared with 73.50% without relation-first memory, a difference of 4.00 percentage points at the same mean core size (8.10) and compression (51.26%). Full also had higher Root@3 (86.25% versus 83.75%) and Risk-any (100.00% versus 98.75%). These aggregate results favor retaining relation-first memory in this comparison.

Removing critical-pair protection and revisiting reduced Risk-all by 21.25 points, to 56.25%, while reducing mean core size from 8.10 to 4.30. This comparison therefore combines changes in evidence selection and output size. Random local windows reduced Risk-all by 6.25 points, to 71.25%, at the same mean core size as Full.

The Stage-1 results are descriptive comparisons across evaluation batches. We report aggregate differences without corresponding confidence intervals or significance tests.

K.4 Stage-2 Paired Core Replay

We replayed frozen GPT-4o Full trajectories for 80 variants from 20 source SCCs without new model calls. Both policies received the same selector proposal at each trajectory prefix. The dynamic policy allowed node replacement; the locked policy admitted nodes up to the cap and retained them without eviction. Both shared OC signals, relation seeding, critical-pair protection, pair closure, thresholds, and the cap. All 80 pairs passed input-identity and final-Full parity checks.

Dynamic replacement increased Risk-all by 37.5 percentage points (95% source-SCC paired interval $[12.5, 60.0]$) and Risk-any by 13.75 points ($[5.0, 22.5]$), at equal mean core size and compression (Table 17). The replay isolates endpoint retention given the observed evidence; search trajectories are fixed, and OC and closure are not ablated independently. The replay and Stage-1 evaluation (Table 16) use different sets of Full outputs on the same 80 variants, yielding Full Risk-all values of 75.00% and 77.50%, respectively.

K.5 Dynamic Core Replacement Across Backbones

We extend paired core replay to Llama 3.1-8B-Instruct, Qwen3.5-9B, and Kimi-VL-A3B, with one Full run per model–case. Each model uses the same 80 injected variants: four templates applied to 20 source SCCs across CUAD, SEC, BGB, and Debian. Each run uses 60 rollouts, four-node windows, internal concurrency four, and temperature 0.4. These 240 trajectories form the ablation cohort, separate from the historical main-experiment results. Tables in this and the next three subsections report percentages averaged over 80 cases per model; the 20 source SCCs are the independent sampling units.

To isolate replacement, both policies receive the same ordered evidence proposals. Dynamic allows earlier admissions to be replaced; locked retains admitted nodes without eviction. For each case, locked capacity is fixed to the final Dynamic output size k, and both return exactly k nodes. This paired replay isolates replacement at fixed evidence and retained-core size. Root@3 uses the shared trajectory ranking and is identical across selectors.

Dynamic replacement increases Risk-all by 27.50, 23.75, and 5.00 percentage points for Llama, Qwen, and Kimi, respectively (Table 18). Risk-any also improves on all three backbones at identical

Model	Method	Root@3	Risk-any	Risk-all	Compression
GPT-4o	LEA	71.25	85.00	41.25	56.61
	Full-SCC	22.50	53.75	23.75	**74.60**
	SA-MCGS	**86.25**	**100.00**	**75.00**	51.26
Qwen2.5-72B	LEA	55.00	**98.75**	50.00	51.26
	Full-SCC	23.75	85.00	46.25	45.32
	SA-MCGS	**70.00**	97.50	**75.00**	**52.82**
Llama 3.1-8B-Instruct	LEA	50.42	93.75	34.58	51.22
	Full-SCC	6.67	55.42	22.50	44.23
	SA-MCGS	**61.25**	**95.00**	**48.75**	**51.30**
Qwen3.5-9B	LEA	58.75	**93.33**	62.08	51.26
	Full-SCC	20.00	63.33	12.08	**71.76**
	SA-MCGS	**62.50**	86.25	**63.75**	67.25
Kimi-VL-A3B	LEA	5.42	**89.58**	17.50	51.26
	Full-SCC	**7.92**	59.58	**52.50**	13.60
	SA-MCGS	5.00	87.50	32.50	**51.85**

Table 15: Five-backbone comparison with SA-referenced baseline budgets on the same 80 variants per model (%). SA-MCGS uses fixed historical outputs. Bold/underline mark the highest/second-highest value within each model, including ties. Shading identifies SA-MCGS; higher compression means a smaller core.

Arm	Root@3 ↑	Any ↑	All ↑	Comp.	$\lvert H\rvert$
80 variants / 20 source SCCs					
Full SA-MCGS	**86.25**	**100.00**	**77.50**	51.26	8.10
No relation-first memory	83.75	98.75	73.50	51.26	8.10
No critical-pair ledger/revisit	83.75	88.75	56.25	71.29	4.30
Random local windows	85.00	96.25	71.25	51.26	8.10

Table 16: Stage-1 ablations with GPT-4o on 80 variants from 20 source SCCs. Root@3, Any (Risk-any), All (Risk-all), and compression (Comp.) are percentages; $|H|$ is mean core size. Bold/underline: best/second-best endpoint metric, including ties; core size and compression are descriptive.

Core policy	Root@3	Any	All	Comp.	$\lvert H\rvert$
Dynamic	**86.25**	**100.00**	**75.00**	51.26	8.10
Locked	**86.25**	86.25	37.50	51.26	8.10

Table 17: Stage-2 replay on shared search trajectories. Rates and compression are percentages. Root@3 is fixed by the shared trajectory and is unaffected by the core policy. Bold/underline: best/second-best endpoint metric (↑), including ties.

Policy	Root@3	Any	All	Comp.
Llama 3.1-8B-Instruct				
Dynamic (Full)	**62.50**	**93.75**	**62.50**	51.26
Locked	**62.50**	85.00	35.00	51.26
Qwen3.5-9B				
Dynamic (Full)	**58.75**	**88.75**	**57.50**	66.55
Locked	**58.75**	70.00	33.75	66.55
Kimi-VL-A3B				
Dynamic (Full)	**7.50**	**93.75**	**35.00**	51.93
Locked	**7.50**	80.00	30.00	51.93

Table 18: Dynamic versus locked core replay at the default capacity ratio $r = 0.45$. Any and All denote Risk-any and Risk-all; Comp. is $100(1 - |H|/|S|)$. Evidence and actual output size are matched per case. Bold/underline mark the best/second-best endpoint rate within each model, including ties; compression is descriptive.

compression. The comparison supports revising the retained core as evidence accumulates, with the largest gains on Llama and Qwen.

K.6 Core Capacity and Retention

We replay the same Full evidence at four capacity ratios, $r \in \{0.25, 0.35, 0.45, 0.60\}$, with cap $\min\{|S|, \max(4, \lceil r|S| \rceil)\}$. At each ratio, we recompute the production selector from the recorded evidence. Locked admission matches Dynamic's actual final k case by case. The cap is an upper bound; observed compression is calculated from the actual retained-core size.

Dynamic retains complete endpoints more often

r	Dynamic	Locked	Δ	Comp.
Llama 3.1-8B-Instruct				
0.25	**25.00**	13.75	+11.25	68.30
0.35	**38.75**	16.25	+22.50	61.76
0.45	**62.50**	35.00	+27.50	51.26
0.60	**75.00**	60.00	+15.00	36.93
Qwen3.5-9B				
0.25	**47.50**	22.50	+25.00	73.49
0.35	**48.75**	26.25	+22.50	70.31
0.45	**57.50**	33.75	+23.75	66.55
0.60	**57.50**	38.75	+18.75	64.21
Kimi-VL-A3B				
0.25	**17.50**	7.50	+10.00	68.34
0.35	**20.00**	13.75	+6.25	61.91
0.45	**35.00**	30.00	+5.00	51.93
0.60	**55.00**	47.50	+7.50	38.67

Table 19: Capacity sweep on the same 80 Full trajectories per model. Dynamic and Locked columns report Risk-all; Δ is their difference in percentage points. Comp. is the shared mean compression of the paired outputs. Bold marks the higher Risk-all within each pair.

than locked admission at all 12 model–capacity settings (Table 19). Dynamic's Risk-all increases with capacity through $r = 0.60$ for Llama and Kimi; Qwen reaches 57.50% at both $r = 0.45$ and 0.60. The paired gains persist across the tested caps, while the absolute coverage and compression quantify the trade-off in output size.

K.7 Selection from Shared Evidence

We compare the Full selector with LEA-score while holding the acquired evidence and final output size fixed. LEA-score applies the membership formula in Appendix K.1 to the recorded Full evaluations and returns its top k nodes, where k is Full's actual output size for that case. Its signals are local-subgraph membership, repair-entry and conflict-endpoint counts, plus mean-risk surplus. The comparison isolates final selection on evidence acquired by SA-MCGS.

Full improves Risk-all over LEA-score by 5.00, 5.00, and 8.75 points on Llama, Qwen, and Kimi (Table 20).

K.8 Evidence Accumulation Across Rollout Prefixes

We reconstruct the native Full core after 15, 30, 45, and 60 completed rollout events from each stored 60-rollout trajectory. These four checkpoints measure evidence accumulation within each trajectory, with the selector, evidence thresholds, and nominal capacity ratio held fixed.

Policy	Root@3	Any	All	Comp.
Llama 3.1-8B-Instruct				
Full	**62.50**	93.75	**62.50**	51.26
LEA-score	**62.50**	**96.25**	57.50	51.26
Qwen3.5-9B				
Full	**58.75**	**88.75**	**57.50**	66.55
LEA-score	**58.75**	86.25	52.50	66.55
Kimi-VL-A3B				
Full	**7.50**	**93.75**	**35.00**	51.93
LEA-score	**7.50**	78.75	26.25	51.93

Table 20: Full versus LEA-score at matched evidence and actual output size, using the default capacity. Endpoint-rate highlighting follows Table 18. Root@3 uses the shared trajectory ranking.

Prefix	Root@3	Any	All	Comp.
Llama 3.1-8B-Instruct				
15	45.00	85.00	51.25	56.39
30	**63.75**	93.75	61.25	51.86
45	57.50	**96.25**	58.75	51.33
60	62.50	93.75	**62.50**	51.26
Qwen3.5-9B				
15	42.50	67.50	33.75	78.23
30	55.00	76.25	45.00	73.19
45	**60.00**	86.25	51.25	69.02
60	58.75	**88.75**	**57.50**	66.55
Kimi-VL-A3B				
15	5.00	73.75	13.75	63.14
30	5.00	83.75	26.25	56.71
45	**7.50**	91.25	31.25	53.03
60	**7.50**	**93.75**	**35.00**	51.93

Table 21: Native Full outputs reconstructed at four rollout prefixes. Each row averages the same 80 cases for its model. Root@3, Risk-any (Any), Risk-all (All), and compression (Comp.) are percentages. Bold/underline identify the highest/second-highest distinct endpoint rate across prefixes within each model; compression describes the evolving output size.

From 15 to 60 events, native Risk-all rises from 51.25% to 62.50% on Llama, 33.75% to 57.50% on Qwen, and 13.75% to 35.00% on Kimi (Table 21). The corresponding cores retain more nodes, quantifying the coverage–compression trade-off as search accumulates evidence. This trajectory analysis complements the paired replacement comparisons at fixed evidence and output size.

K.9 Contemporaneous Cost and Reliability

The cost experiment used one injected variant from each of 12 distinct source SCCs. Debian/SEC/BGB/CUAD contributed 2/3/3/4 SCCs;

Model	Method	Valid	Invalid	Root@3	Any	All	Comp. (n)
GPT-4o	One-shot	9	3	33.3	58.3	25.0	67.2 (9)
	LEA	12	0	83.3	91.7	41.7	51.0 (12)
	SA-MCGS	12	0	**91.7**	**100.0**	**83.3**	51.0 (12)
Gemini 2.5 Pro	One-shot	12	0	**100.0**	**100.0**	**100.0**	69.3 (12)
	LEA	12	0	75.0	**100.0**	83.3	51.0 (12)
	SA-MCGS	12	0	91.7	**100.0**	91.7	57.4 (12)

Model	Method	Calls	Input (K)	Output (K)	Total (K)	Time (s)	
						Mean	Median
GPT-4o	One-shot	**1.0**	**17.0**	**2.9**	**19.9**	**21.0**	**22.1**
	LEA	17.8	36.3	22.8	59.1	189.2	138.0
	SA-MCGS	59.6	112.3	68.4	180.7	570.9	574.7
Gemini 2.5 Pro	One-shot	**1.0**	**19.6**	**7.5**	**27.1**	**85.9**	**66.8**
	LEA	15.9	36.6	31.7	68.3	868.2	443.9
	SA-MCGS	53.5	115.9	110.2	226.1	1650.2	1413.1

Table 22: Cost, performance, and reliability on 12 source SCCs for each model–method pair. All binary rates use a denominator of 12, with invalid outputs counted as failures. Comp. is mean compression over valid outputs; parentheses give the denominator. Calls and tokens include recorded recovery usage and are divided by 12. Tokens are in thousands. Time is recorded runtime per case; parallel speedup was not measured. Within each model, bold/underline mark best/second-best endpoint rates (↑, upper panel) or costs (↓, lower panel); compression and validity are descriptive. Shading identifies SA-MCGS.

each of the four templates contributed three variants; and small/medium/large SCCs numbered 6/3/3. GPT-4o and Gemini 2.5 Pro each ran One-shot, LEA, and SA-MCGS on these frozen inputs, yielding 72 method jobs. SA-MCGS used 60 rollouts, four-node windows, and internal concurrency one. One-shot generated a subgraph directly from the full SCC in a single call, without oracle selection. Table 22 reports the results for each model–method pair.

Of the final outputs, 69 were valid and three were invalid. All invalid outputs came from GPT-4o One-shot and remained in the cost and performance denominators. One Gemini SA-MCGS job was interrupted and restarted, with 17 calls before interruption and 56 during recovery; its valid final output required more than one execution. Recorded usage includes both executions, although costs for unreturned in-flight requests are unknown. Runtime covers the recovery interval, with pre-interruption API time recorded separately.

The paired SA-MCGS–LEA Risk-all differences were $+41.7$ points for GPT-4o (95% interval $[8.3, 75.0]$) and $+8.3$ for Gemini ($[-16.7, 33.3]$). Gemini One-shot achieved the highest endpoint completeness in this sample. Averaged equally across models, SA-MCGS used 203.4K tokens per case, compared with 63.7K for LEA, or approximately $3.2\times$ as many. The performance gains from additional computation thus varied by model.

Internal rollout concurrency can reduce waiting time. A simple approximation to wall-clock time is

$$T_{\text{wall}} \approx T_{\text{serial}} + \sum_g \left\lceil \frac{n_g}{p_g} \right\rceil t_g, \qquad (16)$$

where each dependency-ordered group g contains n_g parallelizable calls, p_g is effective parallelism, and t_g is representative call latency. Available work, workers, and API limits constrain p_g. For fixed work, concurrency leaves calls, tokens, and fees unchanged. Equation 16 models execution time; no within-case speedup was measured, and concurrent execution of separate cases does not establish such a speedup.

K.10 Relation-Prior Sensitivity

We varied the relation-prior bonus multiplier (`relation_prior_weight`) from its default 0.85 to 0.68 and 1.02 ($\pm 20\%$). The four source SCCs (cost slots 01/03/08/10) were selected and frozen before the sensitivity results were observed. Node, edge, and pair feature weights in Table 11 remained fixed. The two models produced 16 new valid outputs; eight nominal outputs with matching inputs and configurations were reused from the cost experiment. All other settings were fixed, including rollout budget and internal concurrency one.

Model	Metric	0.68	0.85	1.02
GPT-4o	Root@3	75	**100**	75
	Risk-all	**100**	75	**100**
Gemini Pro	Root@3	**100**	75	**100**
	Risk-all	**75**	**75**	**75**

Table 23: Exploratory sensitivity to the relation-prior multiplier on four source SCCs per row. Root@3 and Risk-all are percentages; Risk-any is 100% throughout. Bold/underline: highest/second-highest distinct value across settings in each row, including ties.

Endpoint completeness did not decrease on this subset, although root rankings changed (Table 23). This four-cluster check characterizes local sensitivity to one coefficient; it does not establish robustness to other priors or core thresholds. The sensitivity jobs ran concurrently across cases, precluding a direct runtime comparison with the serial nominal runs.

K.11 Identity and Statistical Conventions

Frozen manifests distinguish physical source SCCs, injected variants, and model–method outputs. Inputs are matched by source content, SCC node membership, and injection identity, rather than traversal index alone. Experiment bundles retain raw outputs, result hashes, available prompt/schema and configuration records, parser versions, and call-level usage.

For the ablation and cost experiments, binary performance uses all planned outputs, counting invalid or undefined outcomes as failures. Compression is $1 - |H|/|S|$ and is computed only for defined outputs, with the denominator reported. Risk-any and Risk-all measure intersection and full inclusion of the injected endpoint set in H, respectively, rather than node-level precision. Where intervals are reported, they use 20,000 percentile-bootstrap replicates with seed 8229, resampling physical source SCCs jointly across methods. Source-equal estimates average variants within each SCC before averaging SCCs; variant-level averages are labeled explicitly. Models and variants from the same source do not constitute independent clusters; statistical comparisons must account for such dependence (Dror et al., 2018). The intervals characterize the selected source set, not natural-risk prevalence.

K.12 Main-Experiment Confidence Intervals

For the main experiment, we resampled 20 source SCCs with replacement as paired clusters, keeping each source's four variants, five fixed models, and methods together. Paired differences are averaged within each source over variants and, for the overall estimate, models; source means are then averaged equally. Table 24 reports overall and model-specific 95% percentile intervals from 20,000 bootstrap replicates (seed 8229). These intervals reflect variation across the selected sources rather than repeated-run variability.

K.13 Computational Resources and Implementation Details

Computational resources and budget. Local inference was conducted on a cluster equipped with eight NVIDIA H800 GPUs, with GPU allocation depending on the model. GPT-4o and Gemini 2.5 Pro were accessed through APIs. Across evaluation and development experiments, we estimate aggregate inference usage to be approximately one billion input and output tokens, including recorded retries. Available usage logs account for approximately 0.95 billion tokens; missing historical usage is estimated from comparable cost profiles. Offline analyses reuse existing trajectories without additional model inference.

Software implementation and configuration. The implementation uses Python and NetworkX for graph processing, with QuantLaw for BGB reference extraction. Recorded local deployments use vLLM 0.20.0, PyTorch 2.11.0, and Transformers 5.6.2. The default SA-MCGS configuration uses 60 rollout slots and four-record windows. Evaluation metrics and source-level bootstrap confidence intervals are computed using custom scripts; experiment-specific settings are provided in the corresponding appendix subsections.

Model / scope	Comparator	Root@3	Risk-any	Risk-all
All five (fixed)	LEA	$+6.75$ $[-0.25, 14.00]$	-0.25 $[-4.50, 4.00]$	$+15.25$ $[2.00, 27.75]$
	One-shot	$+46.50$ $[37.75, 55.25]$	$+35.25$ $[24.00, 46.75]$	$+39.00$ $[29.00, 49.25]$
GPT-4o	LEA	$+7.50$ $[-3.75, 20.00]$	$+3.75$ $[0.00, 10.00]$	$+31.25$ $[7.50, 52.50]$
	One-shot	$+60.00$ $[43.75, 75.00]$	$+45.00$ $[27.50, 62.50]$	$+60.00$ $[43.75, 73.75]$
Gemini 2.5 Pro	LEA	$+0.00$ $[-8.75, 7.50]$	-2.50 $[-6.25, 0.00]$	$+0.00$ $[-11.25, 12.50]$
	One-shot	-2.50 $[-16.25, 11.25]$	$+0.00$ $[-5.00, 5.00]$	-1.25 $[-20.00, 16.25]$
Qwen2.5-72B	LEA	$+2.50$ $[-11.25, 16.25]$	$+0.00$ $[-6.25, 5.00]$	$+31.25$ $[10.00, 51.25]$
	One-shot	$+67.50$ $[51.25, 82.50]$	$+51.25$ $[31.25, 71.25]$	$+61.25$ $[43.75, 78.75]$
Llama 3.1-8B-Instruct	LEA	$+17.50$ $[2.50, 32.50]$	$+2.50$ $[-6.25, 12.50]$	$+13.75$ $[-5.00, 32.50]$
	One-shot	$+57.50$ $[41.25, 72.50]$	$+47.50$ $[30.00, 65.00]$	$+23.75$ $[2.50, 45.00]$
Qwen3.5-9B	LEA	$+6.25$ $[-6.25, 20.00]$	-5.00 $[-13.75, 2.50]$	$+0.00$ $[-15.00, 15.00]$
	One-shot	$+50.00$ $[31.25, 68.75]$	$+32.50$ $[18.75, 47.50]$	$+51.25$ $[36.25, 66.25]$

Table 24: Main-experiment SA-MCGS minus comparator differences, in percentage points, with 95% source-SCC bootstrap intervals in brackets.